

Introduzione agli Algoritmi

04/03/2024

Pseudocodice

Per poter valutare il tempo computazione di un algoritmo abbiamo bisogno che questo sia formulato in modo chiaro, sintetico e non ambiguo. Proprio per questo motivo utilizzeremo quello che viene chiamato pseudocodice. In particolare si utilizzano comunque alcuni costrutti di controllo dei linguaggi di programmazione (come for, while, if, else, then etc...). Tuttavia si possono specificare alcune operazioni mentre si omettono certi dettagli per la gestione di errori specifici così da cogliere solo l'essenza vera della soluzione.

Costo delle Istruzioni

Istruzioni Elementari hanno tutti costo $\Theta(1)$

- operazioni matematiche
- lettura del valore di una variabile
- assegnazione di un valore ad una variabile
- valutazione di una condizione logica di una costante di operandi
- stampa del valore

Condizioni IF

Il costo dell'istruzione IF dipenderà da quale ramo dell'IF dovremo navigare. Per rendere le cose più semplici e univoche, andremo a considerare il caso PEGGIORE tra il Then e l'Else. Cioè quale dei due rami impegna più tempo.

Costo IF = costo di verifica della condizione + il caso PEGGIORE tra la fase Then o Else.

Cicli Iterativi

Le istruzioni iterative (cicli) hanno un costo pari alla somma dei costi di ciascuna delle iterazioni (compreso il costo di verifica della condizione). Se tutte le iterazioni hanno lo stesso costo, il costo dell'iterazione è pari al prodotto del costo di una singola iterazione per il numero di iterazioni.

Se invece, le iterazioni non tutte stesso costo andranno sommati tra loro i costi di ogni iterazione.

Accorgimento: La condizione viene valutata una volta in più rispetto al numero delle iterazioni, poiché va considerata anche l'ultima valutazione, che dà esito negativo, è quella che fa terminare l'iterazione (ma se il suo costo è costante, possiamo ignorarlo...).

Valutazione del Costo Computazionale

Il costo dell'algoritmo nel suo complesso è pari alla somma dei costi delle istruzioni che lo compongono.

Un dato algoritmo potrebbe avere tempi di esecuzione diversi, a seconda dell'input. Per questo motivo andiamo a distinguere tra caso migliore e peggiore. Andremo cioè a fissare una dimensione di input, così da identificare un input particolarmente vantaggioso e, rispettivamente, svantaggioso, ai fini del costo computazionale dell'algoritmo.

Ricordando il concetto di O grande e Omega, il caso peggiore limita il costo computazionale da sopra, mentre il caso migliore da sotto. Se questi due valori coincidono possiamo definire il theta.

Esempio: calcolo del massimo

```
def Trova_Max (A):  
    n=len(A)                 $\Theta(1)$   
    max=A[1]                 $\Theta(1)$   
    for i in range (1,n):   (n-1) iteraz. +  $\Theta(1)$   
        if A[i] > max:       $\Theta(1)$   
            max ← A[i]       $\Theta(1)$   
    return max               $\Theta(1)$ 
```

Detto $T(n)$ il costo computazionale di questo algoritmo:

$$\begin{aligned}T(n) &= \underline{\Theta(1)} + \underline{\Theta(1)} + (n-1)*[\Theta(1) + \Theta(1)] + \underline{\Theta(1)} = \\&= \underline{3*\Theta(1)} + (n-1)*[\Theta(1)] = \\&= \underline{\Theta(1)} + \Theta(n-1) = \Theta(n-1)\end{aligned}$$

Esempio: somma dei primi n interi

```
def Calcola_Somma_1 (n):  
    somma = 0                 $\Theta(1)$   
    for i in range (1,n+1):  n iterazioni +  $\Theta(1)$   
        somma += i           $\Theta(1)$   
    return somma             $\Theta(1)$ 
```

$$\begin{aligned}T(n) &= \Theta(1) + n*[\Theta(1)] + \Theta(1) + \Theta(1) = \\&= 3*\Theta(1) + \Theta(n) = \Theta(n)\end{aligned}$$

Osserviamo, tuttavia, che lo stesso problema può essere risolto in modo ben più efficiente attraverso la formula di Gauss:

```
def Calcola_Somma_2 (n):  
    somma = n*(n+1)/2         $\Theta(1)$   
    return somma             $\Theta(1)$ 
```

$$T(n) = \Theta(1) + \Theta(1) = \Theta(1)$$

Esempio: valutazione di un polinomio

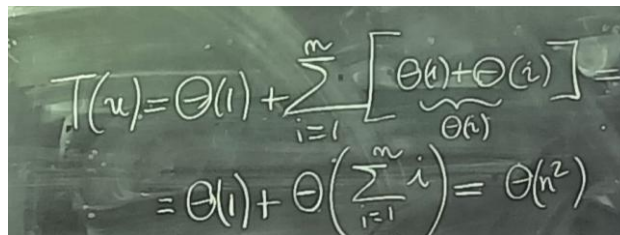
Valutazione del polinomio $\sum_{i=0}^n a_i x^i$ nel punto $x = c$.

Cioè $S = a_0x^0 + a_1x^1 + a_2x^2 + \dots$

```
def Calcola_Polinomio_1(A, c):  
    somma=A[0]                                 $\Theta(1)$   
    for i in range(len(a)):  
        potenza = 1                             $\Theta(1)$   
        for j in range(i):                      $i \text{ iterazioni} + \Theta(1)$   
            potenza = c*potenza                 $\Theta(1)$   
        somma = somma+A[i]*potenza             $\Theta(1)$   
    return somma                                $\Theta(1)$ 
```

Siccome il tempo di esecuzione delle operazioni interne al Primo Ciclo è $\Theta(i)$, questo è un esempio di ciclo in cui il tempo di esecuzione del ciclo dipende dal valore della variabile su cui iteriamo. In questi casi si scrive:

$T(n) = \text{Sommatoria di } i=1 \text{ fino ad } n \text{ di (tutto quello che è nel ciclo)}$


$$\begin{aligned} T(n) &= \Theta(1) + \sum_{i=1}^n [\underbrace{\Theta(1) + \Theta(i)}_{\Theta(n)}] = \\ &= \Theta(1) + \Theta\left(\sum_{i=1}^n i\right) = \Theta(n^2) \end{aligned}$$

Sommatoria Notevole

Riscrivendo in modo più oculato lo pseudocodice, il medesimo problema può essere risolto in modo più efficiente come segue:

```
def Calcola_Polinomio(A,c):  
    somma = A[0]                                 $\Theta(1)$   
    potenza = 1                                 $\Theta(1)$   
    for i in range(1,len(A)):  
        potenza = c*potenza                     $\Theta(1)$   
        somma=somma+A[i]*potenza                 $\Theta(1)$   
    return somma                                $\Theta(1)$ 
```

$$\begin{aligned} T(n) &= \Theta(1) + \Theta(1) + n * [\Theta(1) + \Theta(1)] + \Theta(1) + \Theta(1) = \\ &= 4 * \Theta(1) + n * [\Theta(1)] = \\ &\Theta(1) + \Theta(n) = \Theta(n) \end{aligned}$$

Ricerca Binaria

Ai fini degli esercizi di esame ritornerà MOLTOO utile conoscere la ricerca binario, la quale ha costo $O(\log n)$.

Di seguito il codice in entrambe le versioni, iterativo e ricorsivo.

Ricorsivo

```
def ricercaBinaria(A, key, i, j):  
    if i <= j:        #vado in ricorsione finchè i due indici non si scavalcano  
  
        medio = (i+j)//2    #calcolo l'indice medio dell'array  
        if key < A[medio]:    #se quello che cerco è più piccolo vado a controllare nella parte sinistra dell'array  
            return ricercaBinaria(A, key, i, medio-1)    #cioè in A[i:medio-1] (siccome in A[medio] ho già controllato)  
  
        elif key > A[medio]:    #stessa cosa di qua. Se key > A[medio] vado a controllare nella parte destra dell'array  
            return ricercaBinaria(A, key, medio+1, j)    #cioè in A[medio+1:j]  
  
        return True        #se non è minore e nemmeno maggiore allora A[medio] == key ed ho trovato l'elemento  
  
    return False
```

Iterativo

```
def ricercaBinariaIterativa(A, key):  
    i = 0  
    j = len(A)-1  
    while i <= j:  
        medio = (i+j)//2  
        if key < A[medio]:  
            j = medio-1    #medio escluso perché in A[medio] ho già controllato  
        elif key > A[medio]:  
            i = medio+1    #medio escluso perché in A[medio] ho già controllato  
        elif A[medio]==key:  
            return True  
  
    return False
```

Insertion Sort -> $\Theta(n^2)$

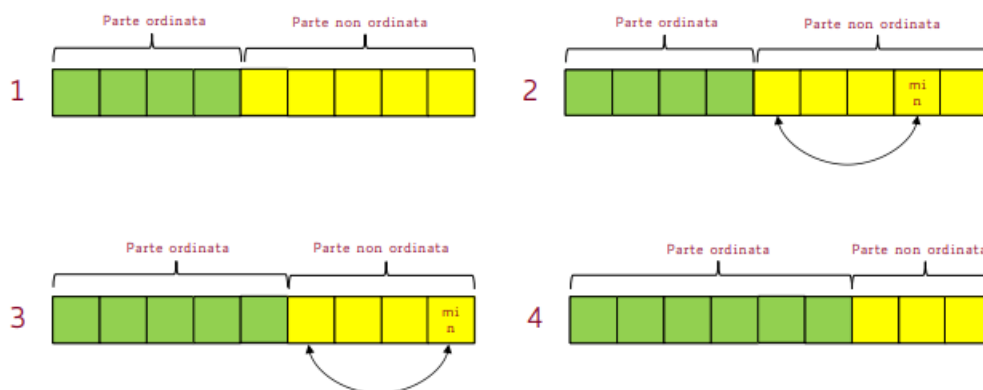
Per rendere subito chiaro il concetto. Immaginiamo di dover ordinare un mazzetto di carte che abbiamo in mano. Prendiamo una carta, la mettiamo nella posizione adeguata (in termini di $>$ o $<$), spostiamo tutte le altre carte.

1. **Prendiamo l'elemento** in indice j . Lo mettiamo in una variabile temporanea x .
2. **Confrontiamo x con gli elementi ALLA SUA SINISTRA** (cioè con quelli da $i-1$ fino a 0):
 - Tutti gli elementi maggiori di x (while $A[j] > temp$) vengono spostati di 1 pos. a destra per fare spazio ad $temp$.
 - Appena trovo $A[i] < temp$ esco dal while e INSERISCO $temp$ nell'array. Di fatto ho trovato l'esatta posizione che soddisfa: $A[j] \leq temp < A[j+2]$

```
def InsertionSort(A):  
    for i in range(1, len(A)):  
        temp = A[i]  
        j = i - 1    #devo controllare gli elementi a sinistra  
        while j >= 0 and A[j] > temp:  
            A[j+1] = A[j]    #sposto l'elemento a destra per fare spazio  
            j = j - 1        #itero sull'elem a sinistra  
        #non appena trovo un elemento A[j] < temp esco dal while  
        A[j+1] = temp #ed INSERISCO temp nella posizione giusta  
        #a questo punto avremo A[j] <= temp < A[j+2]  
    return A
```

Selection Sort -> $\Theta(n^2)$

Cerca il minimo dell'intero array e lo mette in prima posizione (con uno scambio), poi cerca il minimo nell'array restante, e lo mette in seconda posizione (con uno scambio), poi il minimo dell'array restante e lo mette in terza posizione etc...



```
def SelectionSort(A):  
    for i in range(len(A)-1): #per ogni elemento di A  
        m = i                #ipotizzo che il mio elemento sia il minimo  
        for j in range(i+1, len(A)): #vado a cercare il minimo A DESTRA  
            if A[j] < A[m]:      #se trovo qualcosa < minimo  
                m = j           #j diventa il nuovo indice del minimo  
        A[i], A[m] = A[m], A[i] #scambio A[i] con il minimo  
    return A
```

Bubble Sort -> $\Theta(n^2)$

Funziona per fasi. Ogni fase ispeziona una dopo l'altra, da destra a sinistra (QUINDI REVERSED RANGE), ogni coppia di elementi adiacenti e, se l'ordine dei due elementi non è quello giusto, essi vengono scambiati.

In questo modo, ad ogni iterazione l'elemento più piccolo della sequenza che si sta ordinando viene messo in posizione $A[i]$.

```
def BubbleSort(A):  
    n = len(A)  
    for i in range(n-1):  
        for j in reversed(range(i, len(A)-1)): #scorriamo da dx a sx  
            if A[j]>A[j+1]: #se l'ordine è sbagliato  
                A[j], A[j+1] = A[j+1], A[j] #li scambio  
  
    return A
```

Alberi di Decisione (SKIIIIIP)

I tre algoritmi di ordinamento appena visti hanno tutti un costo computazionale asintotico che cresce come il quadrato del numero di elementi da ordinare. Ma si può fare di meglio? Come si fa a stabilire un limite di costo computazionale al di sotto del quale nessun algoritmo di ordinamento può arrivare?

Esiste uno strumento adatto allo scopo, ovvero: l'albero di decisione, che permette di rappresentare tutte le strade che la computazione di uno specifico algoritmo può intraprendere, sulla base dei possibili esiti dei test previsti dall'algoritmo stesso.

Nel caso degli algoritmi di ordinamento basati su confronti, ogni test effettuato ha due soli possibili esiti (essere minore o uguale, oppure no). L'albero di decisione relativo a un qualunque algoritmo di ordinamento basato su confronti ha queste proprietà:

- è un albero binario che rappresenta tutti i possibili confronti che vengono effettuati dall'algoritmo; ogni nodo interno (ossia un nodo che non è una foglia) ha esattamente due figli;
- ogni nodo interno rappresenta un singolo confronto, ed i due figli del nodo sono relativi ai due possibili esiti di tale confronto;
- ogni foglia rappresenta una possibile soluzione del problema, la quale è una specifica permutazione della sequenza in ingresso. (Ciò significa che alle foglie avremo tutte le possibili permutazioni degli elementi iniziali)

Se riusciamo a calcolare l'altezza dell'albero decisionale, avremo una limitazione inferiore al costo computazionale dell'algoritmo (che abbiamo rappresentato nell'albero).

25/03/2024

Osservazione 1: dato che la soluzione può essere una qualunque permutazione della sequenza di ingresso, l'albero di decisione deve contenere nelle foglie tutte le permutazioni della sequenza in ingresso, che sono $n!$ per un problema di dimensione n .

Osservazione 2: un albero binario di altezza h non può contenere più di 2^h foglie.

Segue che l'altezza h dell'albero di decisione di qualunque algoritmo di ordinamento basato su confronti deve essere tale per cui:

$$2^h \geq n! \iff h \geq \log n!$$

e cioè

$$h = \Omega(n \log n)$$

Dimostrazione:

Valutiamo $\log n!$ (per semplicità assumiamo che n sia pari).

$$\begin{aligned} \log n! &= \log(n \cdot (n-1) \cdot (n-2) \cdot (n-3) \dots \cdot 2 \cdot 1) = \\ &= \sum_{i=1}^n \log i = \sum_{i=1}^{n/2} \log i + \sum_{i=\frac{n}{2}+1}^n \log i \geq \\ &\geq 0 + \sum_{i=\frac{n}{2}+1}^n \log \frac{n}{2} = \\ &= \frac{n}{2} \log n - \frac{n}{2} = \Theta(n \log n) \end{aligned}$$

Siccome abbiamo visto che $h \geq \log n!$

e che $\log n! = \Theta(n \log n)$

ciò significa che

$$h = \Omega(n \log n)$$

Quanto detto si riassume nel seguente **Teorema:**

Il costo computazionale di qualunque algoritmo di ordinamento basato su confronti è $\Omega(n \log n)$.

Insertion Sort 2.0 (con Ricerca Binaria)

```
def Bin_Insertion_Sort(A)
    for j in range(1, len(A)):      (n-1)  $\Theta(1)$  +  $\Theta(1)$ 
        x = A[j]                     $\Theta(1)$ 
        k = RicercaBin(A, x)         $O(\log j)$ 
        /* x dovrebbe stare in pos.k
        i = j - 1                     $\Theta(1)$ 
        while ((i >= 0) and (i >= k))  $t_j \Theta(1) + \Theta(1)$ 
            A[i+1] = A[i]              $\Theta(1)$ 
            i = i - 1                  $\Theta(1)$ 
        A[k] = x                      $\Theta(1)$ 
```

Tuttavia questo non ci porta a reali vantaggi. Infatti, nonostante impieghiamo molto meno tempo nella ricerca della posizione in cui mettere il valore, dovremo comunque iterare tutto l'array per spostare gli elementi.

04/04/2024

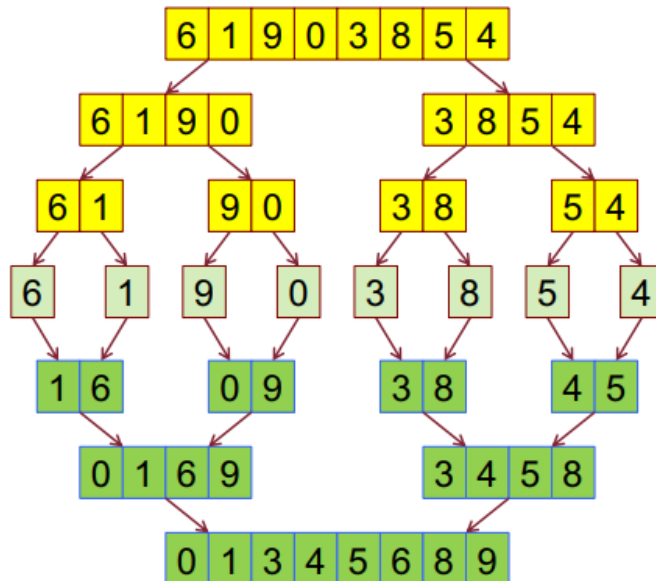
Tecnica "Divide Et Impera"

La Tecnica "Divide Et Impera" può essere divisa in fasi come segue:

- **Divide:** il problema complessivo si suddivide in sottoproblemi di dimensione inferiore
- **Impera:** i sottoproblemi si risolvono ricorsivamente
- **Combina:** le soluzioni dei sottoproblemi si compongono per ottenere la soluzione al problema complessivo

Merge Sort -> $\Theta(n \log n)$

L'algoritmo merge sort è un algoritmo ricorsivo che adotta la tecnica algoritmica Divide Et Impera nel seguente modo:



- **Divide:** l'array da ordinare viene diviso a metà ogni volta, creando due sottosequenze da ordinare.
- **Impera:** richiama ricorsivamente MergeSort su entrambe le sottosequenze. Termina (Caso Base) quando la sequenza da ordinare ha 1 solo elemento. Dunque è già ordinato.
- **Combina:** la parte più complicata. Questa viene eseguita da una funzione ausiliaria solitamente chiamata "fondi" che tramite 2 indici, *i* per scorrere la prima sequenza e *j* per scorrere la seconda, confronta $A[i]$ e $A[j]$, inserisce il più piccolo nell'array ordinato e muove il corrispettivo indice.

Tuttavia il Merge Sort NON permette l'ordinamento "in loco", cioè non può aggiornare direttamente l'array A ma ha bisogno di crearne uno nuovo.

```
def MergeSort(A, ind_primo, ind_ultimo):  
    if ind_primo < ind_ultimo: #se NON sono caso base:  
         $\Theta(1)$  ind_medio = (ind_primo+ind_ultimo)//2 #calcolo ind_medio = len(A)//2  
         $T(n/2)$  MergeSort(A, ind_primo, ind_medio) #ordina A[inizio : ind_medio]  
         $T(n/2)$  MergeSort(A, ind_medio+1, ind_ultimo) #ordina A[ind_medio : fine]  
         $\Theta(n)$  Fondi(A, ind_primo, ind_medio, ind_ultimo) #combinio i risultati  
  
    #Altrimenti se i due indici sono uguali, o uno sorpassa l'altro, la sequenza  
    #ha un solo elemento e termino la ricorsione. (perchè è già ordinato)
```

$$\begin{matrix} T(n)=2T(n/2)+\Theta(n) \\ T(1)=\Theta(1) \end{matrix} \longrightarrow T(n)=\Theta(n \log n)$$

Prendo come parametri l'array A da ordinare, l'indice al primo elemento e l'indice all'ultimo elemento. La prima cosa che faccio è calcolarmi l'indice_medio, cioè il punto centrale dell'array, per andare a dividere in due sottoarray anch'essi da ordinare con MergeSort. Vado quindi in ricorsione finché $\text{ind_primo} < \text{ind_ultimo}$.

Termino quindi quando $\text{ind_primo} = \text{ind_ultimo}$, il che significa che puntano entrambi allo stesso elemento dell'array. In sostanza, significa che il mio array ha un solo elemento ed è, ovviamente, già ordinato.

Fondi -> $\Theta(n)$

```
def Fondi(A, ind_primo, ind_medio, ind_ultimo):
    Ordinato = [] #creo l'array ordinato che ospiterà il risultato

    i = ind_primo #scorro la prima sequenza -> A[ind_primo : ind_medio]
    j = ind_medio+1 #scorro la seconda sequenza -> A[ind_medio+1 : ind_ultimo]
    while i<=ind_medio and j<=ind_ultimo:
        #finchè non termino una delle due sequenze inserisco nell'array
        #ordinato l'elemento piu piccolo
        if A[i]<A[j]:
            Ordinato.append(A[i])
            i = i +1
        else:
            Ordinato.append(A[j])
            j = j +1

    #usciti dal while una delle due sottosequenze risulta non terminata
    #Termino la prima sequenza
    while i<=ind_medio:
        Ordinato.append(A[i])
        i = i +1

    #Termino la seconda sequenza
    while j<=ind_ultimo:
        Ordinato.append(A[j])
        j = j +1

    for k in range(len(Ordinato)): #per ogni elemento che ho ordinato
        A[ind_primo+k]=Ordinato[k] #lo copio nella medesima posizione in A

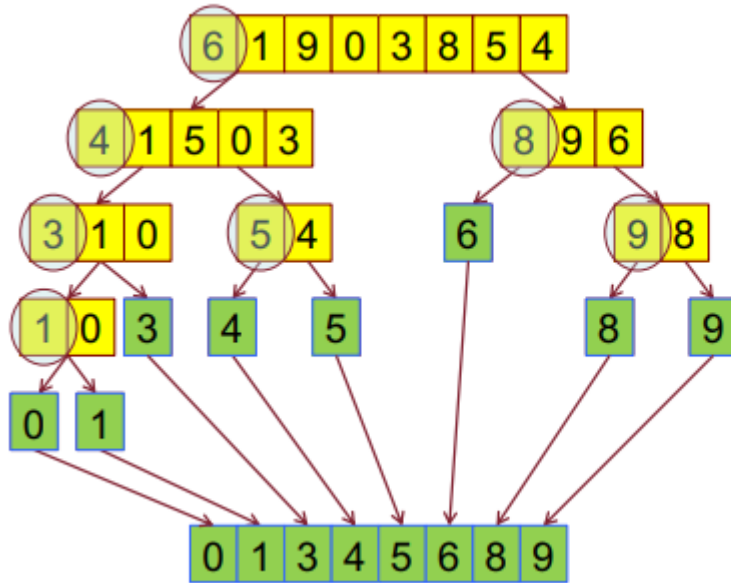
    #NON abbiamo return perchè, in queste due righe, andiamo a modificare
    #direttamente A
```

Quick Sort -> $\Theta(n \log n)$

L'algoritmo QuickSort (ordinamento veloce) ha costo $O(n^2)$ nel caso peggiore ma nella pratica è spesso la soluzione migliore per grandi valori di n perché:

- il suo tempo di esecuzione atteso è $\Theta(n \log n)$;
- i fattori costanti nascosti sono molto piccoli;
- permette l'ordinamento "in loco". (a differenza del Merge Sort)

Riunisce i vantaggi del Selection sort (ordinamento in loco) e del Merge sort (ridotto tempo di esecuzione). Ha però lo svantaggio dell'elevato costo computazionale nel caso peggiore (sebbene si presenti molto raramente). A livello sostanziale anche l'algoritmo QuickSort è un algoritmo ricorsivo che adotta la tecnica algoritmica divide et impera:



- **Divide:** nella sequenza da ordinare viene selezionato un **pivot**. La sequenza viene quindi divisa in 2 sottosequenze: quella degli elementi minori del pivot (a sinistra) e quella degli elementi maggiori uguali del pivot (a destra)
- **Impera:** le due sequenze vengono ordinate ricorsivamente. Si procede finché le sottosequenze sono costituite da un solo elemento
- **Combina:** non occorre, dal momento che il QuickSort consente l'ordinamento in loco.

```

def QuickSort(A, ind_primo, ind_ultimo):
    #il casobase è lo stesso del MergeSort
    if ind_primo < ind_ultimo:
        ind_medio = Partiziona(A, ind_primo, ind_ultimo)
        #calcolo ind_medio dipende da quanti elementi sono < o >= del pivot
        #non possiamo calcolarlo direttamente..

        QuickSort(A, ind_primo, ind_medio)      #ordina A[ind_primo : ind_medio-1]
        QuickSort(A, ind_medio + 1, ind_ultimo) #ordina A[ind_medio+1 : ind_ultimo]

    #la parte di combinazione (es. funz Fondi) non serve nel QuickSort
    #perchè consente l'ordinamento in loco

def Partiziona(A, ind_primo, ind_ultimo):
    pivot = A[ind_primo]      #decidiamo di prendere come pivot il primo elemento
                              #ma potremmo tranquillamente prendere anche l'ultimo
    i = ind_primo             #indice per scorrere l'array normalmente da sinistra a destra
    j = ind_ultimo + 1        #indice per scorrere l'array al contrario da destra a sinistra

    while True:
        # Trova il primo elemento che è maggiore o uguale al pivot
        while True:
            i += 1 #aumento l'indice
            #finche non termino l'array o trovo un elemento >= del pivot
            if i > ind_ultimo or A[i] >= pivot:
                break

        # Trova il primo elemento che è minore o uguale al pivot
        while True:
            j -= 1
            if j < ind_primo or A[j] <= pivot:
                break

        if i >= j: #se i >= j i due indici si sono scavalcati
            break #quindi ho controllato tutti gli elementi e posso uscire

        #Altrimenti scambio gli elementi
        A[i], A[j] = A[j], A[i]

    A[ind_primo], A[j] = A[j], A[ind_primo] #Metto il pivot nella posizione giusta
    return j

```

Costo Computazionale Quick Sort

```

Funzione Quick_sort (A, ind_pr, ind_ult)
    if (ind_pr < ind_ult):
        ind_med=Partiziona(A,ind_pr,ind_ult)
        Quick_sort (A,ind_pr,ind_med)
        Quick_sort (A, ind_med+1,ind_ult)

```

$$T(n)=T(k)+T(n-k)+\Theta(n)$$

$$T(1)=\Theta(1)$$



non sappiamo risolverla con
alcuno dei metodi studiati...

Proviamo allora a ragionare sul codice, per provare a trovare una soluzione su cui applicare il Metodo di Sostituzione. Possiamo facilmente derivare la soluzione per due situazioni, il caso migliore e quello peggiore.

- **Caso migliore:** è quello in cui, ad ogni passo, la dimensione dei due sotto-problemi è identica. L'equazione di ricorrenza diventa: $T(n) = 2T(n/2) + \Theta(n)$ che ha soluzione $T(n) = \Theta(n \log n)$
- **Caso peggiore:** è quello in cui, ad ogni passo, la dimensione di uno dei due sotto-problemi da risolvere è 1. L'equazione di ricorrenza diventa: $T(n) = T(n-1) + \Theta(n)$ che ha soluzione $T(n) = \Theta(n^2)$

Valutiamo ora il costo computazionale nel caso medio, nell'ipotesi che il valore del pivot suddivida con uguale probabilità $1/(n-1)$ la sequenza da ordinare in due sotto-sequenze di dimensioni k ed $n-k$, per tutti i valori di k tra 1 ed $n-1$.

$$T(n) = \frac{1}{n-1} \left[\sum_{k=1}^{n-1} (T(k) + T(n-k)) \right] + \Theta(n)$$

Ora, per ogni valore di i e $k = 1, 2, \dots, n-1$ il termine $T(k)$ compare due volte nella sommatoria, la prima quando $k = i$ e la seconda quando $k = n - i$. Valutiamo dunque il valore di:

$$T(n) = \frac{2}{n-1} \sum_{q=1}^{n-1} T(q) + \Theta(n)$$

Che è l'ipotesi su cui applicheremo il Metodo di Sostituzione.

La risoluzione dell'esercizio è sul quaderno

11/04/2024

Heap Sort -> $O(n \log n)$

L'algoritmo Heapsort è piuttosto complesso ma esibisce ottime caratteristiche:

- come Merge sort ha un costo computazionale di $O(n \log n)$ anche nel caso peggiore
- come Selection sort ordina in loco. Sfrutta una opportuna organizzazione dei dati, ossia una struttura dati, che garantisce una o più specifiche proprietà, il cui mantenimento è essenziale per il corretto funzionamento dell'algoritmo.

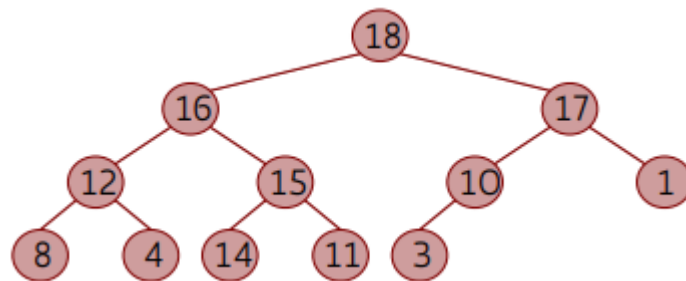
La Struttura dati Heap

La struttura dati heap è stata introdotta per eseguire in modo efficiente una sequenza di operazioni di:

- Estrazione del massimo (cioè la ricerca + rimozione)
- Inserimento e Cancellazione. (mantenendo la struttura dati ordinata)

	Estraz. max	Inserim. di una chiave	Cancellaz. di una chiave
Array qualunque	$O(n)$	$O(1)$	$O(1)$ <i>(Ricerca esclusa!!!)</i>
Array ordinato	$O(1)$	$O(n)$	$O(n)$
heap	$O(\log n)$	$O(\log n)$	$O(\log n)$

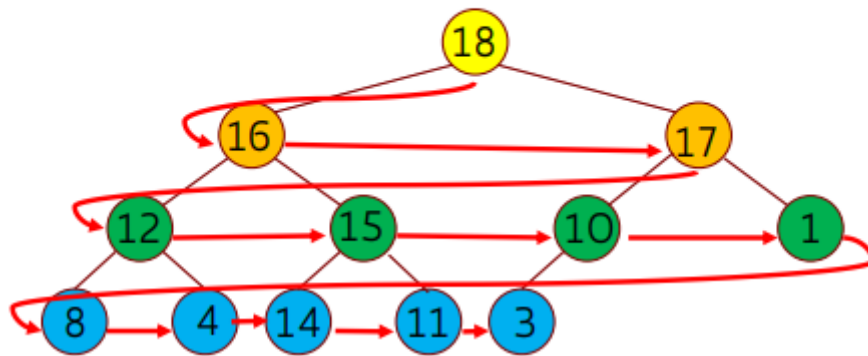
Un heap è un albero binario completo o quasi completo in cui ogni nodo è maggiore o uguale al valore dei suoi figli.
(proprietà di ordinamento verticale)



Avendo la proprietà di essere un albero binario completo o quasi completo la sua altezza (che chiameremo "heap_size") sarà sempre $\Theta(\log n)$. Il modo più naturale per memorizzare un heap è utilizzare un array i cui elementi sono messi in corrispondenza con i nodi dell'heap:

- **L'array è riempito a partire da sinistra**; se contiene più elementi del numero heap_size di nodi dell'albero, allora i suoi elementi di indice $> \text{heap_size}$ non fanno parte dell'heap;
- **Ogni nodo** dell'albero binario corrisponde a **uno e un solo elemento** dell'array A
- **Il figlio sinistro** del nodo A[i], se esiste, corrisponde all'elemento $A[2i+1]$: $\text{left}(i) = 2i+1$;
- **Il figlio destro** del nodo A[i], se esiste, corrisponde all'elemento $A[2i+2]$: $\text{right}(i) = 2i+2$;
- **Il padre del nodo** che corrisponde ad A[i] corrisponde all'elemento $A[(i-1)/2]$: $\text{parent}(i) = \lfloor (i-1)/2 \rfloor$

0.	1.	2.	3.	4.	5.	6.	7.	8.	9.	10.	11.
18	16	17	12	15	10	1	8	4	14	11	3



Proprietà dello Heap

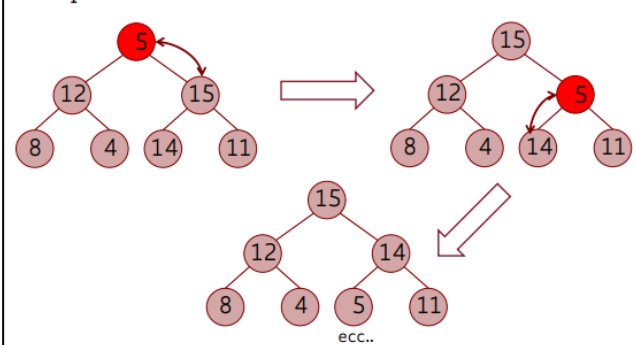
- Poiché lo heap ha tutti i livelli completamente pieni tranne al più l'ultimo, la sua altezza è $\Theta(\log n)$.
- Con l'implementazione tramite array, la proprietà di ordinamento verticale implica che per tutti gli elementi tranne $A[0]$ (che non ha genitore) vale: $A[i] \leq A[\text{parent}(i)]$.
- L'elemento massimo risiede nella radice dell'albero, cioè in $A[0]$, e quindi può essere trovato in tempo $O(1)$.

Funzione Ausiliaria Heapify $O(\log n)$

La funzione Heapify mantiene la proprietà di ordinamento verticale, sotto l'ipotesi che nell'albero da sistemare sia garantita la proprietà di heap per entrambi i sottoalberi (sinistro e destro) della radice. Di conseguenza, l'unico nodo che può violare la proprietà di heap è la radice dell'albero, che può essere minore dei figli.

La funzione opera sulla radice confrontandola coi suoi figli e, se necessario, la scambia col maggiore dei suoi figli. Dopo lo scambio si verifica se la violazione si sia trasferita sul figlio scambiato e, se necessario, si ripete ricorsivamente l'operazione su tale nodo.

Esempio:



```
def Heapify (A, i, heap_size)
    L=left(i); R=right(i); indice_max=i
    if ((L < heap_size) and (A[L] > A[i])):
        indice_max=L
    if ((R <= heap_size) and (A[R] >
        A[indice_max]))
        indice_max=R
    if (indice_max != i)
        A[i], A[indice_max]=A[indice_max], A[i]
        Heapify (A, indice_max, heap_size)
```

$T(h)$

$\Theta(1)$

$T(h-1)$

Prendo l'indice al figlio sinistro ($L=2i+1$), l'indice al figlio destro ($R=2i+2$) ed inizializzo $\text{indice_max} = i$, nel caso in cui la radice del mio heap sia già il massimo. A questo punto iniziano i vari controlli:

- se il figlio sinistro ($A[L]$) è maggiore della radice potrebbe essere il massimo
- se il figlio destro ($A[R]$) è maggiore del presunto massimo diventa il nuovo massimo
- se nessuno dei due figli è maggiore del padre, indice_max è ancora uguale ad i , e lascio tutto così com'è perché abbiamo già verificata la proprietà dell'heap
- altrimenti effettuo lo scambio tra figli e padre per portare il max alla radice

Siccome ad ogni livello, al massimo svogliamo un confronto ed uno scambio (entrambe operazioni costanti da $\Theta(1)$) il costo di questa funzione sarà uguale all'altezza dell'albero. Cioè $T(n) = O(\log n)$.

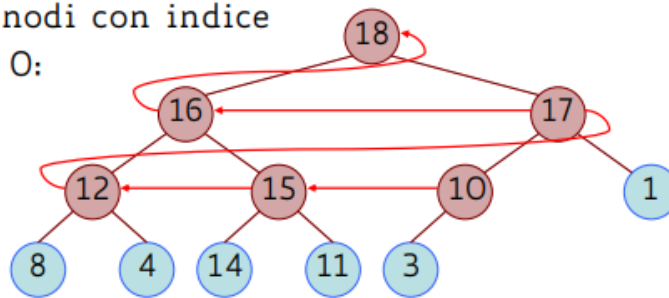
Funzione Ausiliaria Buildheap $O(n)$

La funzione Buildheap trasforma un array qualunque di n elementi in uno heap. Per fare ciò è sufficiente chiamare Heapify su tutti i nodi interni dell'albero (escludendo le foglie). Inoltre:

- Poiché Heapify assume che entrambi i sottoalberi della radice siano heap, va chiamata scorrendo l'albero per livelli dal basso verso l'alto; Cioè **da destra verso sinistra** quando è un array.
- Ogni foglia è già uno heap, perché non avendo figli è automaticamente già più grande di loro, quindi basta chiamare Heapify a partire dal nodo interno più a destra, che ha indice $\lfloor n/2 \rfloor$.

```
def BuildHeap(A):  
    #è sufficiente applicare heapify sui nodi interni dell'albero  
    #escludendo le foglie (che negli alberi binari completi  
    #o semi-completi hanno indice len(A)//2))  
    for i in reversed(range(len(A)//2)):  
        Heapify(A, i, len(A))
```

In questo esempio, Buildheap chiama
Heapify sui nodi con indice
5, 4, 3, 2, 1, 0:



La funzione Build_heap effettua $\Theta(n)$ chiamate di Heapify, che sappiamo avere ciascuna costo $O(\log n)$, quindi inizialmente può sembrare che il costo sia $T(n) = O(n \log n)$. In realtà, con un calcolo più accurato, si può mostrare che il costo computazionale è: **$T(n) = \Theta(n)$**

Funzionamento Heap Sort

• Fase 1:

1. Trasforma un array A di dimensione n in un heap (con $\text{heap_size} = n$), mediante Buildheap(A).
2. Ora, per le proprietà dell'heap, il max dell'array è in $A[0]$ e, per metterlo nella corretta posizione dell'ordinamento, basta metterlo in ultima posizione non ordinata, $A[n-1]$

• Fase 2:

1. La dimensione dell'heap viene ridotta ad $(n - 1)$, siccome l'ultimo elemento è sicuramente il più grande;
2. I due sottoalberi, destro e sinistro, della radice sono ancora degli heap;
3. Solo la nuova radice (ex foglia più a destra) può violare la proprietà del nuovo heap; (non ho capito)
4. Ripristina la proprietà di heap applicando Heapify sui residui $(n - 1)$ elementi;
5. Riapplica il procedimento riducendo via via la dimensione dell'heap fino ad arrivare a $n = 2$;

```
def HeapSort(A, heap_size):  
     $O(n)$  BuildHeap(A)    #Prima di tutto mi trasformo l'array qualunque in un Heap  
     $(n-1)$  volte: for i in reversed(range(1, len(A))):  
         $\Theta(1)$   $A[0], A[i] = A[i], A[0]$     #metto il più grande in ultima posizione  
         $O(\log n)$  Heapify(A, 0, i)    #ripristino l'heap rimanente.  
                                     #cioè  $A[:i]$   
  
    #uscito da questo ciclo for A risulterà come un heap ordinato perchè ogni  
    #iterazione prendo l'elemento più grande, lo metto nella posizione giusta  
    #(cioè in fondo all'array) e ripristino l'heap.
```

$$T(n) = O(n) + (n-1)(\Theta(1) + O(\log n)) = O(n \log n)$$

heapify_bottom_up() -> $O(\log n)$

Per alcune operazioni sugli Heap, come l'inserimento o la rimozione di un elemento specifico, la funzione Heapify NON è sufficiente a ripristinare l'heap. Infatti essa parte da un nodo e aggiusta l'heap verso il basso.

Pensiamo dunque al caso in cui vogliamo inserire un nuovo elemento. Esso dovrà ovviamente essere aggiunto in coda, come foglia dell'heap, e successivamente aggiustata.

Tuttavia, se proviamo ad aggiustarla con Heapify applicata su quella foglia non ci saranno cambiamenti (in quanto Heapify confronta con i figli che, in quanto foglia, non ha). Ugualmente se applicassimo Heapify sulla radice dell'albero non accadrebbe nulla, perché la radice e i suoi figli soddisfano già la proprietà.

Quello che dobbiamo fare è dunque creare una nuova funzione ausiliaria **heapify_bottom_up()** che confronta il nuovo nodo inserito con il padre ed, eventualmente, li scambia. In questo modo navighiamo l'heap dal basso verso l'alto ricercando la posizione giusta in cui inserire l'elemento.

```
'ITERATIVA'
def heapify_bottom_up(A, i):
    while i > 0:
        parent = (i-1)//2
        if A[i] > A[parent]:
            A[i], A[parent] = A[parent], A[i]
            i = parent
        else:
            break
```

```
'RICORSIVA'
def heapify_bottom_up(A, i):
    if i == 0:
        return
    parent = (i - 1) // 2
    if parent >= 0 and A[i] > A[parent]:
        A[i], A[parent] = A[parent], A[i]
        heapify_bottom_up(A, parent)
```

Inserimento -> $O(\log n)$

```
def add_to_heap(A, elem, heap_size):
    A.append(elem)
    heap_size += 1
    ind_ultimo = heap_size - 1
    heapify_bottom_up(A, ind_ultimo)
```

Rimozione -> $O(n)$

```
def remove_from_heap(A, elem, heap_size):
    i = A.index(elem)
    ind_ultimo = heap_size - 1

    #scambio l'elemento da eliminare mettendolo in ultima posizione
    A[i], A[ind_ultimo] = A[ind_ultimo], A[i]

    #Rimuovo effettivamente l'elemento
    A.pop()
    heap_size = heap_size - 1

    #Riaggiusto l'heap
    Heapify(A, i, heap_size)
    heapify_bottom_up(A, i)
```

15/04/2024

Algoritmi di Ordinamento Lineari

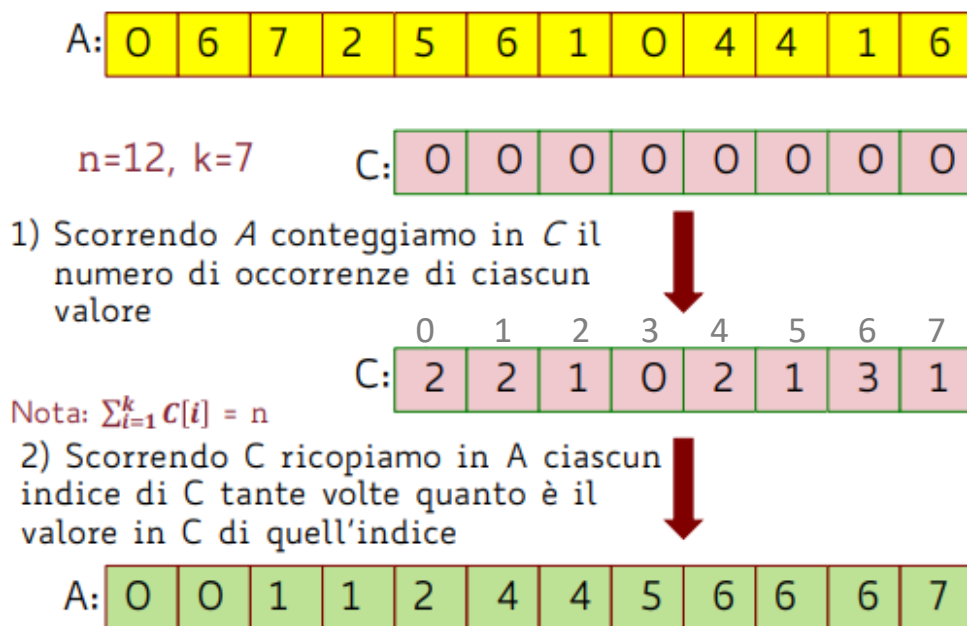
Abbiamo dimostrato il teorema che asserisce che ogni algoritmo di ordinamento che opera per confronti ha un costo computazionale di $\Omega(n \log n)$. Dunque, com'è possibile avere degli "algoritmi di ordinamento lineari", di costo computazionale lineare? Evitando confronti e facendo ipotesi aggiuntive.

Counting Sort -> $\Theta(n+k)$

Ordina gli elementi sfruttando il conteggio, cioè contando il numero di occorrenza di ogni elemento. Il costo computazionale è di $\Theta(n+k)$. Se $k = O(n)$ allora l'algoritmo ordina n elementi in tempo lineare, cioè $\Theta(n)$.

L'idea è quella di fare in modo che il valore di ogni elemento della sequenza determini direttamente la sua posizione nella sequenza ordinata SENZA fare confronti. È fondamentale che ciascuno degli n elementi da ordinare è un intero di valore compreso in un intervallo $[0..k]$ (si può variare leggermente...)

Esempio:



```
def CountingSort(A):
    #per rappresentare i numeri che vanno da 0 a x
    #ho bisogno di un array lungo x+1
    indice_max = max(A)+1
    C = [0]*indice_max      #inizializzo C come array di tutti zero
    for i in range(len(A)): #per ogni elemento di A
        C[A[i]] += 1        #mi segno il numero di occorrenze in C

    #mi vado ora ad ordinare A tramite il numero di occorrenze corrispondenti
    #ad ogni numero in C
    j = 0
    for i in range(len(C)-1): #scorro gli elementi di C
        while C[i]>0: #se C[i] compare ALMENO 1 volta in A
            A[j] = i  #lo scrivo in A
            j += 1    #più volte di fila
            C[i] -= 1 #tante volte quante sono le sue occorrenze

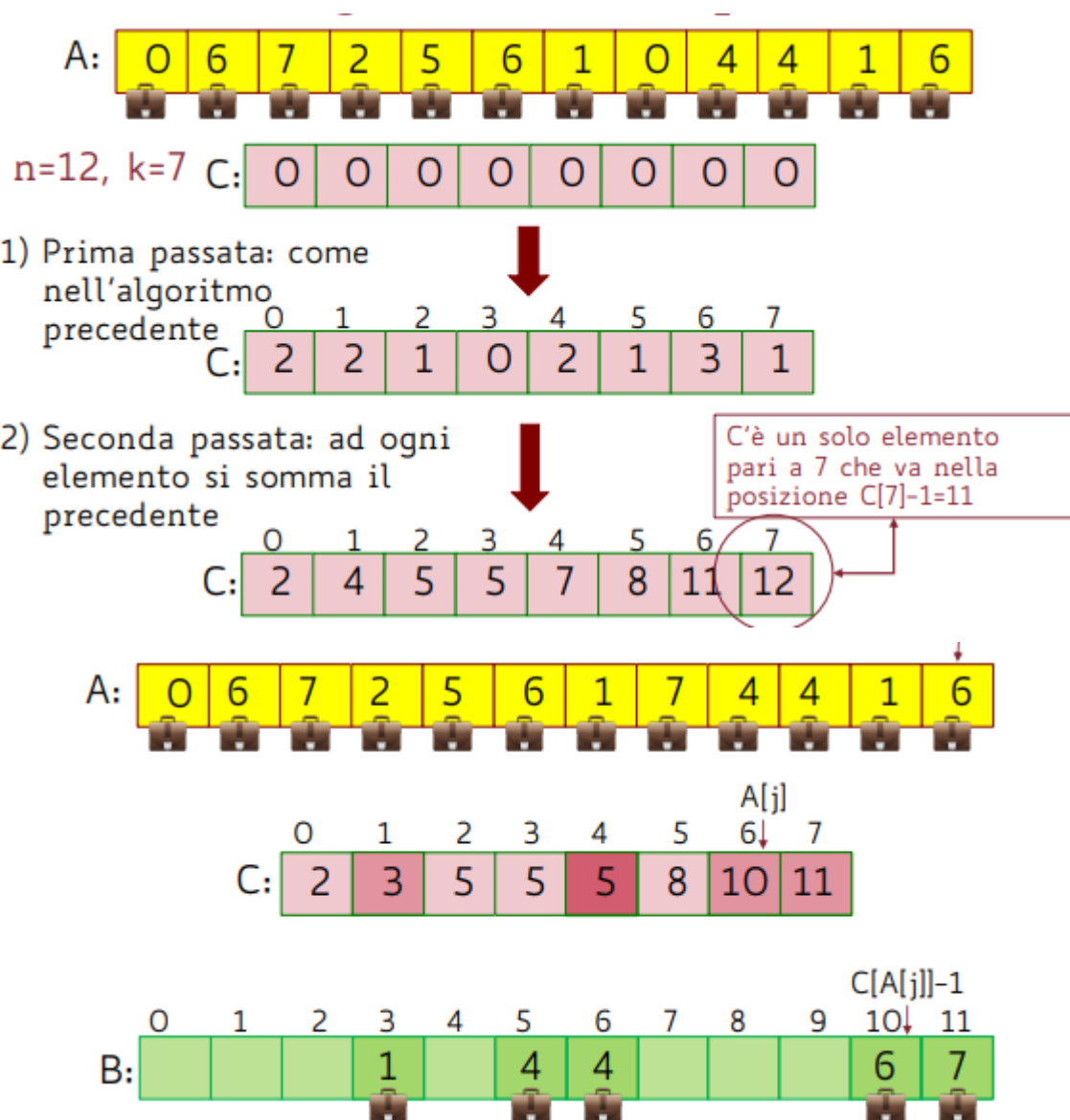
    #A=[2,4,1,1,1] => C=[0,3,1,0,1] => A=[1,1,1,2,4]
```

$$T(n) = \Theta(k) + \Theta(n)$$

Tuttavia, lo pseudocodice appena illustrato è adeguato solamente se non vi sono dati satelliti. Cioè dati correlati tra loro. (immaginiamo per esempio le matricole degli studenti con correlati gli esami sostenuti ed i voti. Se riordiniamo le matricole dovremo portarci appresso tutti gli altri dati satelliti collegati).

Infatti, il ciclo che riscrive il vettore A di fatto ne sovrascrive i dati. Se ci sono dati satellite, oltre agli array A e C si deve introdurre un nuovo array B di n elementi, che alla fine conterrà la sequenza ordinata, e si deve usare uno schema leggermente più complesso... Si conteggiano gli elementi di A in C, si fa una seconda passata su C, nella quale a ogni elemento C[i] si somma il precedente;

- A questo punto: – C[i] indica la posizione corretta, nell'array ordinato, per l'elemento di valore i più a destra; – C[i] - C[i-1] indica quanti elementi ci sono con valore i
- si scorre quindi l'array A da destra a sinistra e, per ogni elemento A[j]: – si copia in B A[j]=k nella posizione giusta, che è C[k]; – si decrementa di 1 il valore C[k].



Nuovo Pseudocodice:

```
Funzione Counting_sort_con_Dati_Satellite (A)
  k=max(A); n=len(A)
  C[0]*(k+1); B=[0]*n
  for j in range(n)
    C[A[j]]+=1      //in C[i] ora c'è il # di elem. = i
  for i in range(1,k)
    C[i]+=C[i-1]    //in C[i] ora c'è il # di elem. ≤ i
  for j in range(n,-1)
    B[C[A[j]]-1]=A[j]
    C[A[j]]-=1
  return B
```

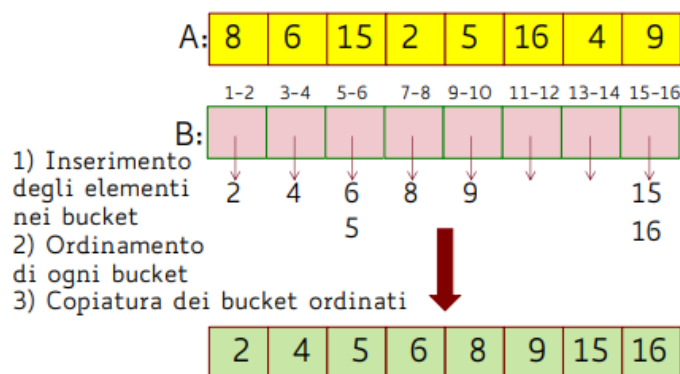
il cui costo è chiaramente $T(n) = \Theta(k) + \Theta(n)$

Bucket Sort -> $\Theta(n)$

Per far funzionare il Bucket Sort avremo bisogno che gli n elementi da ordinare siano **distribuiti in modo uniforme** nell'intervallo $[1..k]$, senza alcuna ipotesi su k . Il costo computazionale medio è di $\Theta(n)$ e l'idea generale è quella di dividere l'intervallo $[1..k]$ in n **sotto-intervalli detti bucket**, tutti di dimensione k/n , e distribuire i valori nei bucket **SENZA** confrontare gli elementi tra loro.

Poiché gli elementi in input sono uniformemente distribuiti, non ci si aspetta che molti elementi cadano nello stesso bucket.

$n=8, k=16$



Pseudocodice:

```
Funzione Bucket_Sort (A; k, n: interi)
  for i=1 to n
    inserisci A[i] nella lista B[ ]          n  $\Theta(1)$ 
  for i=1 to n
    ordina la lista B[i] con insertion_sort
                                          $\sum_{i=1}^n (\text{costi di ordinamento})$ 
  concatena le liste B[1] B[2] ... B[n]
                                         in quest'ordine  $\Theta(n)$ 
  Copia la lista unificata in A               $\Theta(n)$ 
```

I costi di ordinamento dipendono dalla lunghezza delle liste. Ci sono n bucket ed n valori; per l'ipotesi di equidistribuzione, in media ci sarà un numero costante di valori in ogni bucket, quindi la lista $B[i]$ ha in media lunghezza costante. Ciò significa che in media $T(n) = \Theta(n)$.

Strutture Dati

In un linguaggio di programmazione:

- **Un Dato** è il valore che una variabile può assumere;
- **Un Tipo di Dato Astratto** è un modello matematico, dato da una collezione di valori (campi) ed un insieme di operazioni ammesse su questi valori (metodi).
- **Le Strutture Dati** sono particolari tipi di dato, caratterizzate dall'organizzazione dei dati, più che dal tipo di dato stesso.

Una struttura dati è quindi composta da un modo sistematico di organizzare i dati ed un insieme di operatori che permettono di manipolare la struttura stessa. Le strutture dati possono essere:

- Lineari o Non Lineari (se gli elementi sono messi in maniera sequenziale o meno)
- Statiche o Dinamiche (se possono o no variare la dimensione nel tempo)
- Omogenee o Disomogenee (rispetto ai dati contenuti)

Insiemi Dinamici

Gli elementi dell'insieme dinamico (solitamente indicato come "record") possono essere complessi e contenere più di un dato "elementare". In tal caso distinguiamo:

- **Chiave Univoca**, utilizzata per distinguere gli elementi, il cui valore fa parte di un insieme totalmente ordinato (es. la matricola per gli studenti);
- **Dati Satellite**, cioè ulteriori dati relativi all'elemento stesso ma non direttamente utilizzati nelle operazioni di manipolazione dell'insieme dinamico. (es nome, cognome di uno studente che possono variare da istanza ad istanza)

Le principali operazioni che si compiono sugli insiemi dinamici (che si suppone totalmente ordinabile) possono essere Operazioni di Interrogazione (Query) o Operazioni di Modifica della Struttura Dati.

Tipiche Operazioni di Interrogazione

- **Search(S, k)**: recupera l'elemento con chiave k, se è presente in S, restituisce un valore speciale nullo altrimenti;
- **Min(S)/Max(S)**: recupera il minimo/massimo valore presente in S;
- **Predecessor(S, k)/Successor(S, k)**: recupera l'elemento presente in S che precederebbe/seguirebbe quello di valore k (di cui supponiamo di conoscere la locazione) se S fosse ordinato;

Tipiche Operazioni di Modifica

- **Insert(S, k)**: inserisce un elemento di valore k in S;
- **Delete(S, k)**: elimina da S l'elemento di valore k (di cui supponiamo di conoscere la locazione).

Le differenti strutture dati che vedremo hanno tutte la capacità di memorizzare insiemi dinamici ma differiscono fra loro, anche profondamente, per le proprietà che le caratterizzano. Queste proprietà sono determinanti nella scelta quando si progetta un algoritmo. Un esempio di struttura dati lo abbiamo già incontrato: la struttura dati Heap, senza la quale l'algoritmo Heapsort non potrebbe nemmeno essere pensato.

Arrays

Prima di procedere allo studio di alcune nuove strutture dati, vediamo il costo computazionale delle principali operazioni su insiemi dinamici memorizzati su array, disordinati o ordinati.

Principali Operazioni su Array

Search(S,k)

- array disordinato: bisogna scorrere l'array: $O(n)$.
- array ordinato: ricerca binaria: $O(\log n)$

Min(S), Max(S)

- array disordinato: bisogna scorrere l'array: $\Theta(n)$.
- array ordinato: primo o ultimo elemento: $\Theta(1)$

Predecessor(S,k), Successor(S,k)

- array disordinato: bisogna scorrere l'array: $\Theta(n)$.
- array ordinato: elemento precedente o seguente: $\Theta(1)$

Insert(S,k)

- array disordinato: inserimento nella prima posizione libera: $\Theta(1)$.
- array ordinato: ricerca della posizione, scorrimento a destra degli elementi maggiori ed inserimento: $O(n)$

Delete(S,k)

- array disordinato: eliminazione e scambio con l'ultimo elemento: $\Theta(1)$.
- array ordinato: eliminazione e scorrimento a sinistra per coprire lo spazio lasciato: $O(n)$.

Riassumendo

Struttura dati	Search(S,k)	Minimum(S) Maximum(S)	Predecessor(S,k) Successor(S,k)	Insert(S,k)	Delete(S,k)
Array qualunque	$O(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$O(1)$
Array ordinato	$O(\log n)$	$\Theta(1)$	$\Theta(1)$	$O(n)$	$O(n)$

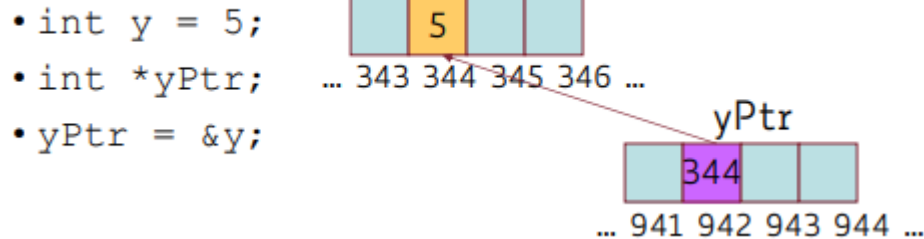
Già da questo semplice confronto, si intuisce che non ha senso dire che una struttura è migliore di un'altra.

È subito evidente che strutture dati possono essere più o meno adatte ad un certo algoritmo e sta al buon progettista disegnare un algoritmo efficiente usando la struttura dati più adatta.

Liste Puntate

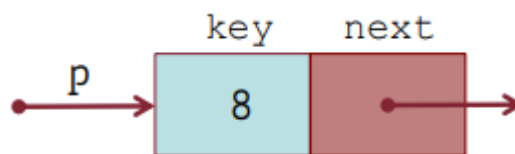
Per introdurre le Liste Puntate è importante sapere cos'è un puntatore. Un puntatore è una variabile che assume come valore un indirizzo di memoria. Il nome di una variabile è un riferimento diretto ad un valore, mentre un puntatore lo fa indirettamente.

Esempio:



Ogni elemento di una lista puntata è un record a due campi:

- **Campo Key:** contiene l'informazione vera e propria;
- **Campo Next:** contiene il puntatore all'elemento successivo; nel caso dell'ultimo elemento della lista contiene un apposito valore None (o null) visualizzato col simbolo “\”

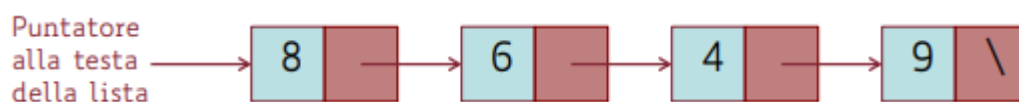


Nello pseudocodice utilizzeremo la sintassi:

- **p->key** indica il contenuto del campo key dell'elemento puntato da p.
- **p->next** indica il contenuto del campo next dell'elemento puntato da p, ossia l'indirizzo dell'elemento successivo (o None se esso è l'ultimo elemento).

La lista puntata semplice è quindi una struttura dati in cui gli elementi sono organizzati in successione. Proprietà specifiche delle liste sono:

- L'accesso avviene sempre ad una estremità, per mezzo di un puntatore alla testa della lista;
- Ogni elemento contiene un puntatore che consente l'accesso al solo elemento successivo;
- È permesso solo un accesso sequenziale agli elementi. Non è possibile accedere direttamente all'elemento in indice x.

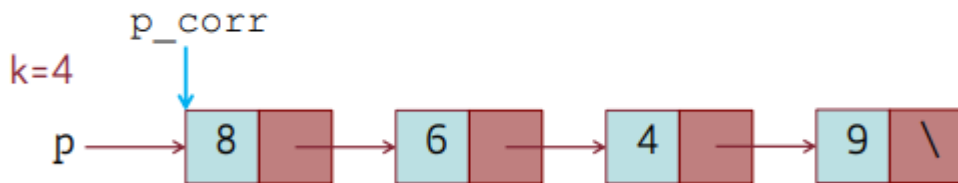


Questo implica che nelle liste puntate semplici l'accesso a qualsiasi dato ha costo proporzionale alla sua posizione, nel caso peggiore $\Theta(n)$.

Principali Operazioni su Liste Puntate

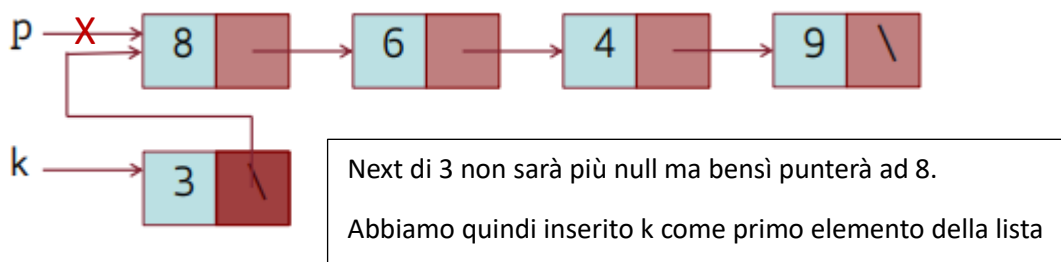
Ricerca

```
Funzione Search (p: puntatore alla lista; k: valore)
    p_corr = p
    while ((p_corr ≠ NULL) and (p_corr->key ≠ k))
        p_corr = p_corr->next
    return p_corr
```



Il costo computazionale della ricerca è $O(n)$.

Inserimento



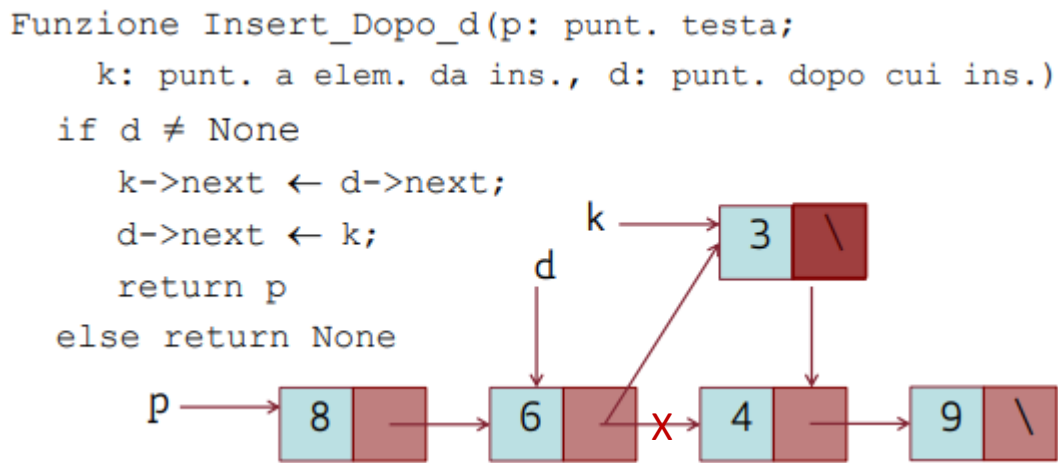
Per inserire un nuovo elemento, senza andare a modificare nessun altro puntatore, conviene decisamente inserire il nuovo elemento come primo. Cioè come testa della lista. Per farlo avremo quindi bisogno che il puntatore iniziale sia k (e non più p) e che il next del nuovo elemento punti a p (cioè il vecchio primo elemento).

```
Funzione Insert_in_testa (p: puntatore alla lista;
                          k: punt. all'elem. da inserire)
    if (k ≠ None)
        k->next = p
    p = k
    return p
```

Il costo computazionale dell'inserimento è $\Theta(1)$.

Nella funzione `Insert_in_testa` abbiamo preso come parametro il puntatore k all'elemento da inserire. In generale, questo non ci viene passato ma bisogna crearlo!!! Questo è il trucco per far sì che la lista diventi una struttura dati dinamica. Tale creazione avviene tramite una allocazione di memoria.

Allora, oltre ad inserire il nuovo dato in testa, potremmo anche pensare di inserire in una posizione qualsiasi, ad esempio dopo la posizione indicata da un puntatore:



Se d non è null (cioè se c'è effettivamente un elemento successivo) faccio puntare il successivo di k al successivo di d, e sovrascrivo il successivo di d facendolo puntare a k.

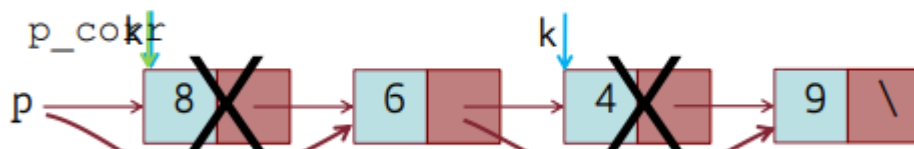
Tuttavia è importante precisare che se invertissimo le righe NON avremmo lo stesso risultato. Pertanto FARE ATTENZIONE all'ordine con cui scambiamo i puntatori.

Eliminazione

```

Funzione Delete (p: punt. alla lista;
    k: puntat. all'elem. da cancellare)
if (k ≠ None) // se k=NULL non c'è niente da cancellare
    if k = p // cancel. 1° elem
        p = p->next; return p
    p_corr = p
    while (p_corr-> next ≠ k)
        p_corr = p_corr-> next // qui, p_corr punta
                                //all'elem. che precede k
    p_corr-> next = k-> next
    return p

```



Il costo computazionale della ricerca è $O(n)$.

Dopo aver specificato un “caso base” nel caso in cui devo eliminare il primo elemento, ragioniamo iterando su ogni elemento e controllando il suo successivo. Quando troveremo un puntatore che ha come successivo k, allora ci fermiamo. Spostiamo il puntatore facendo in modo che k venga “saltato”, rimuovendolo dalla Lista (anche se la sua locazione in memoria rimarrà comunque, senza però possibilità di essere raggiunta).

L'operazione opposta all'allocazione è quella che libera la memoria fisicamente, oltre che logicamente. Nei moderni linguaggi di programmazione viene fatta automaticamente, ogni qual volta una zona di memoria non sia più referenziata da alcun puntatore

Così come la ricerca, non posso accedere direttamente all'elemento che voglio eliminare, ma dovrò fare il tutto sequenzialmente. Quindi avremo costo $O(n)$.

Eliminazione Ricorsiva

Come notato fino ad ora, i procedimenti sono sempre simili tra loro, ed ogni record strutturalmente identico al precedente. Pertanto possiamo dire che le Liste Puntate sono inerentemente ricorsive. Quindi proviamo a modificare il codice dell'eliminazione per renderlo ricorsivo:

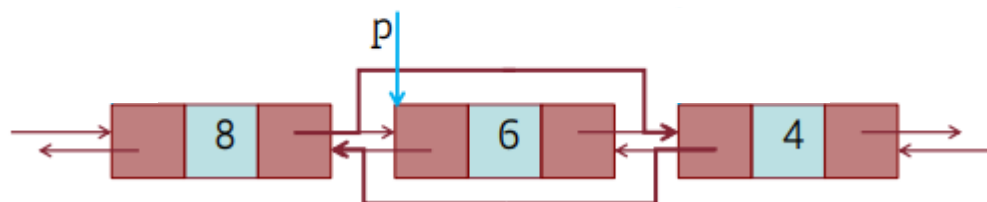
```
Funzione Delete_Ric(p: punt. alla lista;  
                  k: puntat. all'elem. da canc.)  
    if p==k  
        p=p->next  
    else  
        p->next=Delete_Ric(p->next, k)  
    return p
```

Quando lavoriamo ricorsivamente per modificare le liste (inserimenti, eliminazioni etc.) sarà sempre importante avere **p->next = chiamata ricorsiva** altrimenti rischiamo di fare casino con i puntatori non interessati da spostamenti.

Liste Doppiaemente Puntate

In alcuni casi tornava particolarmente utile conoscere, oltre l'elemento successivo, anche il precedente. Pertanto è stata creata quest'altra struttura dati, cioè la Lista Doppiaemente Puntata. In particolare osserviamo come fare la cancellazione:

```
... (p-> prev) -> next = p-> next  
(p-> next) -> prev = p-> prev ...
```



Riassumendo

Struttura dati	Search(S,k)	Minimum(S) Maximum(S)	Predecessor(S,k)	Insert(S,k)	Delete(S,k)
Lista semplice	$O(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$O(n)$
Lista doppia	$O(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$

Digressione sull'oggetto list del quick

L'oggetto list ha somiglianze e differenze sia con gli array che con le liste semplici. Infatti:

- Come si fa negli array:
 - è possibile accedere ai suoi elementi tramite la loro posizione;
 - la sua lunghezza è nota in tempo costante tramite il comando len.
- Come si fa nelle liste semplici:
 - è possibile memorizzare dati non omogenei;

- la lunghezza non è necessariamente fissa.

L'implementazione di questo oggetto è fatta collegando tramite puntatori (elemento per elemento) una lista semplice ad un array. Per quanto riguarda la lunghezza variabile delle list questa si ottiene perché le list vengono create come array di lunghezza 50. Se aggiungendo elementi si sforassero questi 50 elementi verrebbe creata un'altra list (composta sempre di array+lista semplice) collegata alla precedente.

Costo computazionale delle operazioni semplici:

- **.append** (inserim. in ultima posizione): $\Theta(1)$;
- **.insert** (inserim. in i-esima posizione): $O(n)$ poiché bisogna far scorrere in avanti tutti gli elementi in posizione successiva alla i-esima;
- **.pop** oppure del (elimina e restituisce oppure elimina l'elem. in i-esima posiz.): $O(n)$ poiché bisogna far scorrere indietro gli elementi in posizione successiva;
- **.concatenate** (unifica due oggetti di tipo list in uno solo): $\Theta(k)$ dove k è la lunghezza del secondo oggetto.

FARE ESERCIZII!!!!!!

Lo Stack – La Pila

La Pila, o Stack, è una struttura dati con comportamento **LIFO** (Last In First Out). Su una pila sono definite solo due operazioni: **Inserimento (Push)** ed **Estrazione (Pop)**. Non è previsto altro modo per scandire o eliminare elementi con mezzi diversi da Push o Pop.

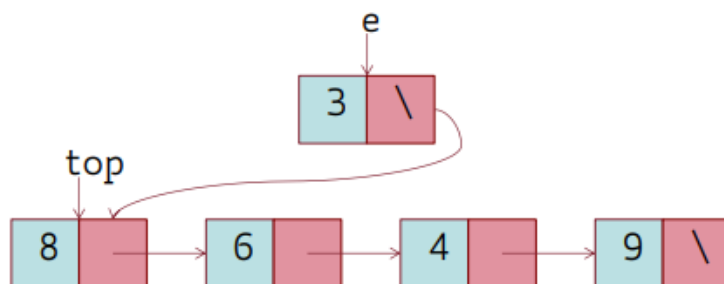
Particolarità della proprietà LIFO: le operazioni Push e Pop operano sulla stessa estremità della pila (attraverso il puntatore top).

Ma, a livello pratico, come possiamo implementare le pile?

Implementazione con Liste

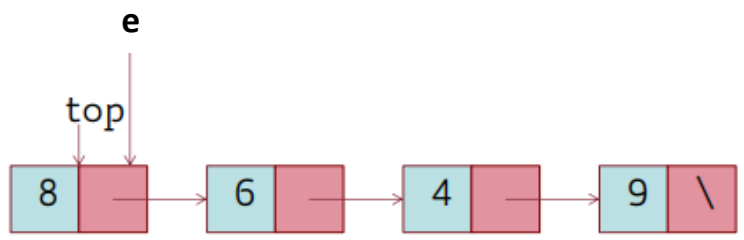
Inserimento: Push

```
fun Push(top: punt.; e: punt. all'elem. da inserire)
    e->next= top
    top = e
    return top
```



Estrazione: Pop

```
fun Pop (top: puntatore)
    if (top==None) //la coda è vuota
        scrivi "Errore: coda vuota" e return None
    e = top
    top = e->next
    e->next=None
    return e, top
```



Il costo computazionale di entrambe le operazioni, Push e Pop, è $\Theta(1)$

Implementazione con Arrays

Le pile possono essere realizzate anche mediante arrays, ma SOLO SE il numero massimo di elementi è noto a priori (visto che gli array sono di lunghezza statica!).

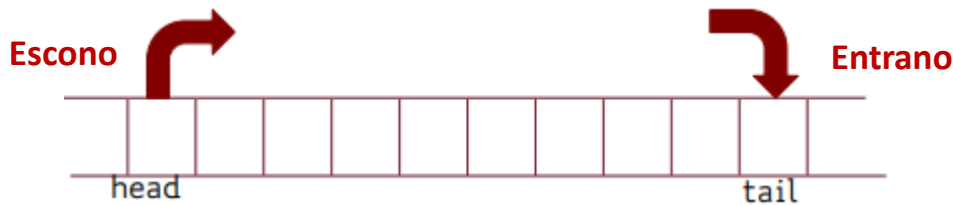
Problema: dobbiamo mantenere il costo delle funzioni Push e Pop costante, per cui non va bene inserire e cancellare in prima posizione. Soluzione: il top della pila dovrà essere posizionato in corrispondenza dell'elemento più a destra, tramite un opportuno indice (non un puntatore!) all'ultima posizione dell'array occupata.

Queue – La Coda

La coda è una struttura dati con comportamento **FIFO** (First In First Out). In altre parole, la coda ha la proprietà che gli elementi vengono da essa prelevati esattamente nello stesso ordine col quale vengono inseriti.

Esempio: la coda di stampa, in cui documenti mandati in stampa prima vengono stampati prima.

Esattamente come le Pile, su una Coda sono definite solo due operazioni: **Inserimento (Enqueue)** ed **Estrazione (Dequeue)**. Non è prevista la scansione degli elementi né l'eliminazione con mezzi diversi da questi. Per la proprietà FIFO, Enqueue opera su una estremità della coda (tail) e Dequeue sull'altra (head).

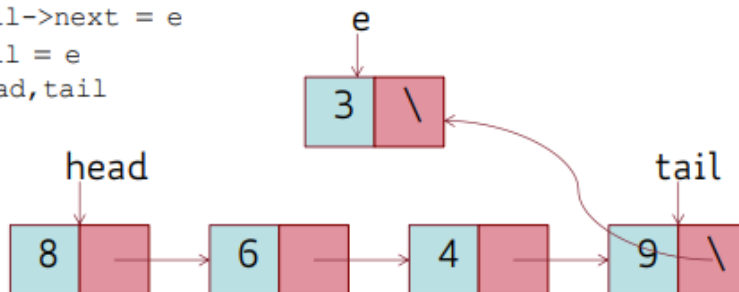


Sebbene sia poco “logico” che gli elementi entrino a destra ed escano a sinistra questo ci ritorna molto più comodo nello svolgimento delle funzioni. In particolare con l'estrazione (Dequeue) in quanto per estrarre il primo elemento (con le liste) è molto semplice e si può fare in $\Theta(1)$. Se dovessimo invece cancellare l'ultimo dovremmo scorrere sequenzialmente tutta la lista!!! Che richiede costo $\Theta(n)$.

Implementazione con Liste

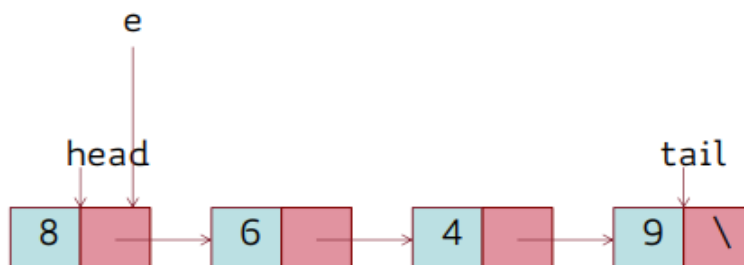
Inserimento: Enqueue

```
Funzione Enqueue (head: puntatore; tail: puntatore;  
                  e: punt. all'elem. da inserire)  
    if (tail == NULL) //la coda è vuota  
        tail = e  
        head = e  
    else  
        tail->next = e  
        tail = e  
    return head, tail
```



Estrazione: Dequeue

```
Funzione Dequeue (head: puntatore; tail: puntatore)  
    if (head == None) //la coda è vuota  
        scrivi "Errore: coda vuota" e return None  
    e = head  
    head = e->next  
    e->next = None  
    if (head == None)  
        tail = None  
    return head, tail, e
```



Implementazione con Arrays

Le code possono essere realizzate anche mediante arrays se il numero massimo di elementi è noto a priori.

Problema: a seguito di ripetute operazioni di Enqueue e Dequeue, gli elementi della coda si spostano via via verso una delle due estremità dell'array e, quando la raggiungono, sembra non esserci più spazio per i successivi inserimenti.

Soluzione: gestire l'array in modo circolare, considerando cioè il primo elemento come successore dell'ultimo.

02/05/2024

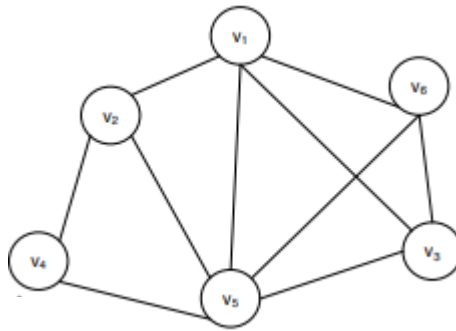
Alberi

L'albero è una struttura dati estremamente versatile, utile per modellare una grande quantità di situazioni reali e progettare le relative soluzioni algoritmiche. Per dare la definizione formale di albero è necessario prima fornire alcune definizioni relative ad un'altra struttura dati, il grafo:

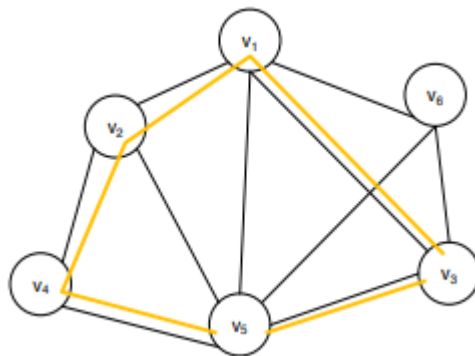
Grafo

Definizione: Un grafo $G = (V, E)$ è costituito da una coppia di insiemi:

- un insieme finito V dei nodi, o vertici;
- un insieme finito E sottoinsieme di $V \times V$ di coppie non ordinate di nodi, dette archi.



Un cammino in un grafo $G = (V, E)$ è una sequenza di nodi distinti di V connessi a due a due. Se il primo e ultimo nodo coincidono possiamo definire un ciclo. Un grafo G è aciclico se non contiene cicli.

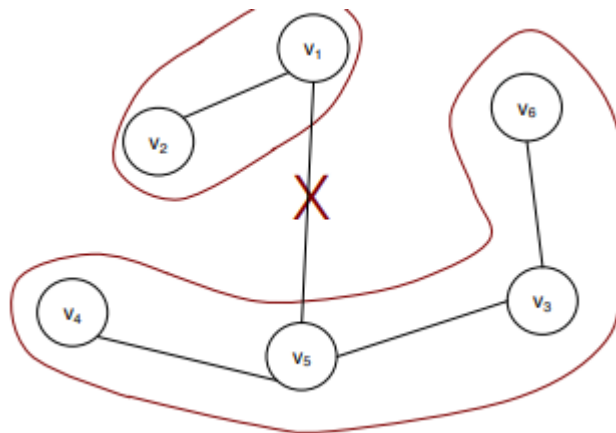


Un grafo G viene detto **connesso** se, per ogni coppia di nodi (u, v) , esiste un cammino tra u e v .

Albero

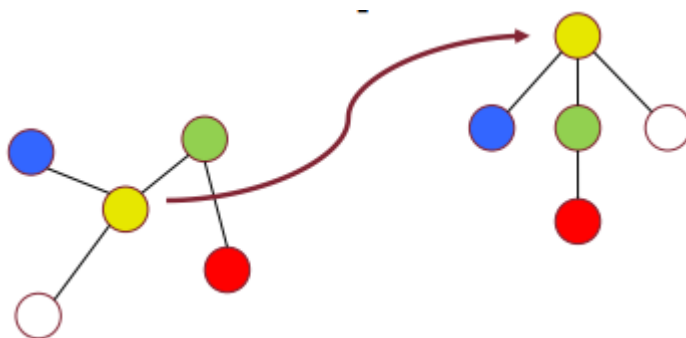
Definizione. Un **Albero** è un grafo $G = (V, E)$ connesso e aciclico

Lemma: Sia $G=(V,E)$ un grafo connesso aciclico (cioè un Albero) eliminando da G un arco qualsiasi, G si disconnette in due grafi connessi e aciclici $G_1 = (V_1, E_1)$ e $G_2 = (V_2, E_2)$



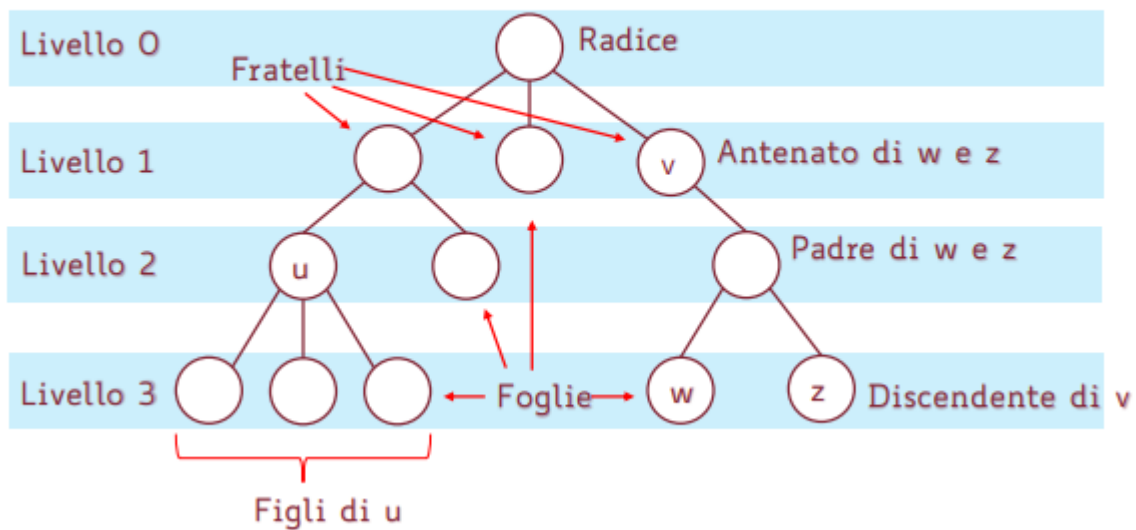
Alberi Radicati

Negli Alberi Radicati vi si distingue un nodo particolare tra gli altri, detto radice. L'albero radicato si può rappresentare in modo tale che i cammini da ogni nodo alla radice seguano un percorso dal basso verso l'alto, come se l'albero venisse, in qualche modo, "appeso" per la radice.



- i nodi sono organizzati in livelli, numerati in ordine crescente allontanandosi dalla radice (di norma la radice è posta a livello zero)
- l'altezza è la lunghezza del più lungo cammino dalla radice ad una foglia; un albero di altezza h contiene $(h + 1)$ livelli, di norma numerati da 0 ad h
- Dato un qualunque nodo v che non sia la radice, il primo nodo che si incontra sull'unico cammino da v alla radice viene detto padre di v
- nodi che hanno lo stesso padre sono detti fratelli e la radice è l'unico nodo che non ha padre
- ogni nodo sul cammino da v alla radice viene detto antenato di v
- tutti i nodi che ammettono v come padre sono detti figli di v , ed i nodi che non hanno figli sono detti foglie
- tutti i nodi che ammettono v come antenato vengono detti discendenti di v

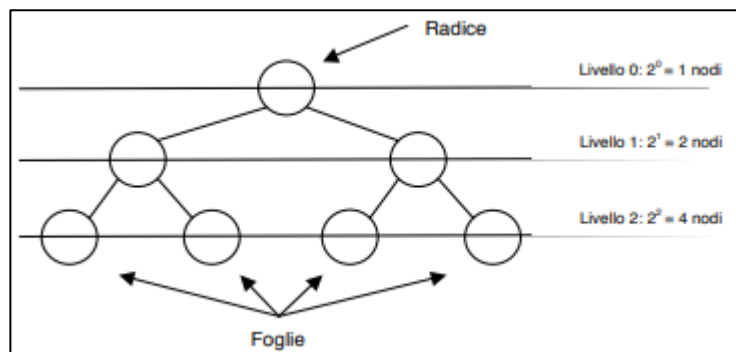
Alberi radicati (esempio, $h = 3$)



Alberi Radicati Ordinati

Un Albero Radicato si dice **Ordinato** se attribuiamo un qualche ordine ai figli di ciascun nodo. Una particolare sottoclasse di alberi radicati e ordinati è quella degli alberi binari, che hanno la particolarità che ogni nodo ha al più due figli. Poiché sono alberi ordinati, i due figli di ciascun nodo si distinguono in figlio sinistro e figlio destro.

Un albero binario nel quale tutti i livelli contengono il massimo numero possibile di nodi è chiamato albero binario completo. Se invece tutti i livelli tranne l'ultimo contengono il massimo numero possibile di nodi mentre l'ultimo livello è riempito completamente da sinistra verso destra solo fino ad un certo punto, l'albero è chiamato albero binario quasi completo.



Inoltre (Sappiamo già che) in un **Albero Binario Completo** di altezza h :

- il numero delle foglie è 2^h
- il numero dei nodi interni è $2^h - 1$
- il numero totale dei nodi è $2^{h+1} - 1$
- L'altezza $h = \log(n + 1) - 1 = \log((n + 1)/2)$

Foglie totali = (numero foglie + numero nodi interni)

$$2^h + 2^h - 1 = 2^{h+1} - 1$$

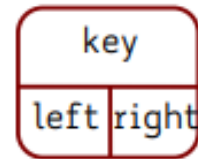
In un albero binario NON Completo ...

Rappresentazione in memoria

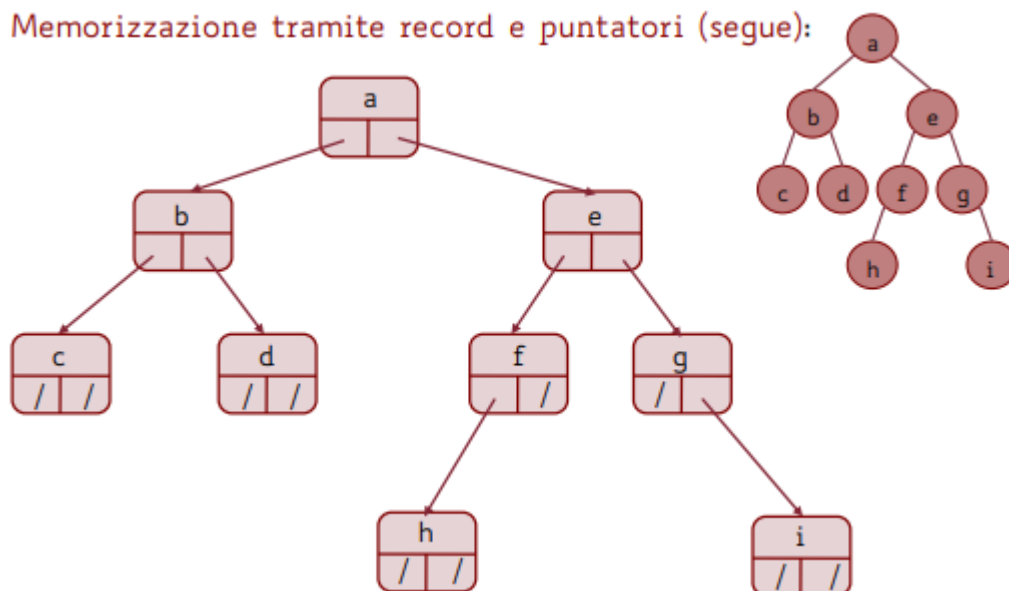
Memorizzazione tramite record e puntatori

Ogni nodo è costituito da un record contenente:

- **key**: le opportune informazioni pertinenti al nodo stesso;
- **left**: il puntatore al figlio sinistro (oppure None se il nodo non ha figlio sinistro);
- **right**: il puntatore al figlio destro (oppure None se il nodo non ha figlio destro);



Memorizzazione tramite record e puntatori (segue):

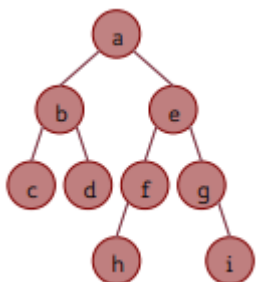


L'albero viene acceduto per mezzo del puntatore alla radice

Memorizzandolo in questo modo l'Albero avrà tutti i **vantaggi** e l'elasticità delle strutture dinamiche basate sui puntatori (si possono inserire nuovi nodi, spostare dei nodi, ecc) ...ma ne presenta gli **svantaggi** moltiplicati: l'unico modo per accedere all'informazione memorizzata in un nodo è scendere verso di esso partendo dalla radice e spostandosi di padre in figlio... ..ma non è chiaro se ad ogni passo si debba andare verso il figlio sinistro o verso il destro...

Rappresentazione Posizionale

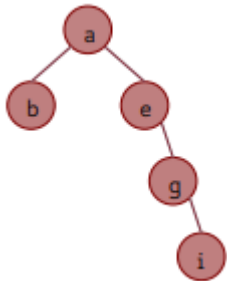
Già discusso in merito allo heap: I nodi vengono memorizzati in un array, nel quale la radice occupa la posizione di indice 0 ed i figli sinistro e destro del nodo in posizione i si trovano rispettivamente nelle posizioni $2i+1$ e $2i+2$.



0.	1.	2.	3.	4.	5.	6.	7.	8.	9.	10.	11.	12.	13.	14.
a	b	e	c	d	f	g	-	-	-	-	h	-	-	i

Svantaggi rispetto alla gestione mediante puntatori:

- richiede di conoscere in anticipo la massima altezza h dell'albero e, una volta noto tale valore, richiede l'allocazione di un array in grado di contenere un albero binario completo di altezza h (quindi un array contenente $2^{h+1} - 1$ elementi);
- Nel caso di un nodo vuoto, saremo obbligati a lasciare uno spazio vuoto come elemento di un array. Dunque, a meno che l'albero non sia abbastanza "denso" di nodi, si verifica uno spreco di memoria.



10.	11.	12.	13.	14.										
a	b	e	-	-	g	-	-	-	-	-	-	-	-	i

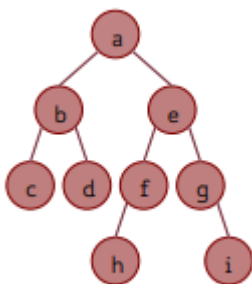
Vantaggi: è una rappresentazione molto comoda da gestire, ma solo nel caso di alberi completi o quasi completi, in cui non c'è questo grande spreco di memoria.

Esercizio. Quant'è la massima lunghezza di un array capace di memorizzare qualsiasi albero con n nodi nella rappresentazione posizionale?

Soluzione: Un albero di n nodi ha, nel caso peggiore, altezza $(n-1)$; Per memorizzare un albero di altezza h con la notazione posizionale è necessario predisporre un array di $2^{h+1} - 1$ elementi;

Rappresentazione con Vettore dei Padri

Costituito da un array P , di lunghezza n , in cui ogni elemento è associato ad un nodo dell'albero. Introducendo una biiezione tra gli n nodi dell'albero e gli indici $0, \dots, n-1$, $P[i]$ contiene l'indice del padre del nodo i nell'albero. Questo metodo di memorizzazione funziona senza alcuna modifica anche per alberi non necessariamente binari, in cui cioè ogni nodo può avere un numero qualunque di figli.



	0	1	2	3	4	5	6	7	8
array delle chiavi →	a	b	c	d	e	f	g	h	i
array P →	/	0	1	1	0	4	4	5	6

Confronto tra Rappresentazioni

Trovare il Padre di un Nodo

Struttura a Puntatori: avremmo bisogno di un puntatore che ci dice chi è il padre. Ma non ce lo abbiamo. Quindi al momento non siamo in grado di farlo (ad ogni passo, destra o sinistra?).

Rappresentazione Posizionale: il padre del nodo i è banalmente in posizione $\left\lfloor \frac{i-1}{2} \right\rfloor$ ed ha costo $\Theta(1)$

Vettore dei Padri: L'indice del padre di ogni nodo i è memorizzato direttamente in $P[i]$, quindi lo possiamo fare in tempo costante accedendo alla posizione i dell'array. Costo $\Theta(1)$

Determinare il numero di Figli (0, 1 o 2 figli)

Struttura a puntatori: si verifica se i campi left e right siano settati a None oppure no: costo $\Theta(1)$

Rappresentazione Posizionale: si vedono i figli in posizione $[2i+1]$ e $[2i+2]$. Se sono settati a '-' oppure no: costo $\Theta(1)$

Vettore dei Padri: Si scorre l'array P e si conta il numero di occorrenze di i: costo $\Theta(n)$

Determinare la Distanza di un Nodo dalla Radice

Struttura a puntatori: non lo sappiamo fare: visite.

Rappresentazione Posizionale: calcolare la distanza consiste nel determinare a che livello ci troviamo. Il livello del nodo i (e quindi la sua distanza dalla radice) è $\lceil \log(i + 1) \rceil$: costo $\Theta(1)$

Vettore dei Padri: a partire dalla posizione i risaliamo di padre in padre passando per $P[i]$, $P[P[i]]$, $P[P[P[i]]]$, ecc. fino alla radice. Costo: nel caso peggiore stiamo cercando la distanza dall'ultimo nodo (foglia) e quindi dobbiamo risalire tutto l'albero $\Theta(h)$

FARE ESERCIZI!!!!!!

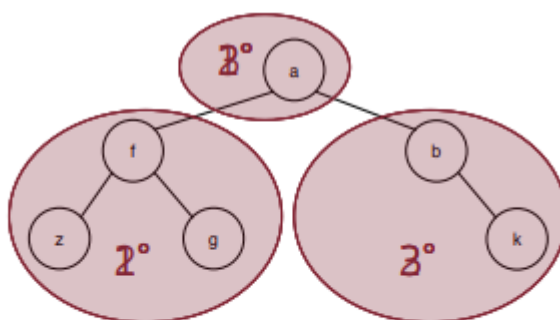
06/05/2024

Visite di Alberi

L'accesso progressivo a tutti i nodi di un albero si chiama visita dell'albero.

Facendo riferimento all'ordine col quale si accede ai nodi dell'albero, è evidente che esiste più di una possibilità. Nel caso degli alberi binari, nei quali i figli di ogni nodo (e quindi i sottoalberi) sono al massimo due, e volendo comportarsi nello stesso modo su tutti i nodi, abbiamo tre diverse visite:

- **visita in pre-ordine (pre-order)**: il nodo è visitato prima di proseguire la visita nei suoi sottoalberi;
- **visita in-ordine (in-order)**: il nodo è visitato dopo la visita del sottoalbero sinistro e prima di quella del sottoalbero destro;
- **visita post-ordine (post-order)**: il nodo è visitato dopo entrambe le visite dei sottoalberi.



visita in preordine:	a f z g b k
visita in inordine:	z f g a b k
visita in postordine:	z g f k b a

Memorizzazione tramite record e puntatori

```
def Visita_preordine (p):  
    if (p != None)  
        accesso al nodo e operazioni conseguenti  
        inordine visita_preordine (p->left) postordine  
        visita_preordine (p->right)  
    return
```

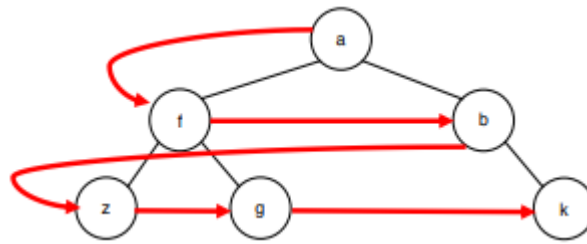
Le altre visite sono analoghe. L'unica differenza fra i tre casi è la posizione dell'istruzione relativa all'accesso al nodo per effettuarvi le operazioni desiderate.

Per il calcolo del costo computazionale guardare il quaderno

IMPORTANTE

OGNI VOLTA che ci viene chiesto di riprodurre/modificare un algoritmo per la visita di un albero BISOGNA DIMOSTRARE il costo computazione con metodo di sostituzione nel modo in cui abbiamo fatto sul quaderno.

Visita per Livelli



Nessuna delle visite ricorsive che abbiamo illustrato per gli alberi implementati mediante puntatori permette di farlo... È necessario utilizzare una coda d'appoggio, nella quale inserire opportunamente i nodi, estraendoli poi per visitarli. Per semplicità, supponiamo che l'implementazione della coda sia fatta in modo tale da inserire ed estrarre direttamente puntatori a nodi dell'albero.

```
def Visita_per_livelli (r; head, tail):
    if (r==None): return
    Enqueue(head, tail, r)
    while (!CodaVuota(head))
        p = Dequeue(head, tail)
        print(p->key)
        if (p->left!=None): Enqueue(head, tail, p->left)
        if (p->right!=None): Enqueue(head, tail, p->right)
    return
```

Sostanzialmente, inserisco in ordine da sinistra a destra, i nodi di ogni livello. Dopo di che rimuovo un elemento dalla coda e lo stampo. Controllo se ha figli, in tal caso li inserisco (sempre in ordine) nella coda.

Dizionari

Un dizionario è una struttura dati che permette di gestire un insieme dinamico di dati, che di norma è un insieme totalmente ordinato, tramite queste tre sole operazioni:

- **Insert:** si inserisce un elemento;
- **Search:** si ricerca un elemento;
- **Delete:** si elimina un elemento.

Assunzioni e Nomenclatura

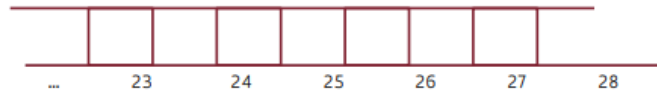
- U: insieme universo dei valori che le chiavi possono assumere; U è costituito da valori interi;
- m: numero delle posizioni a disposizione nella struttura dati;
- n: numero degli elementi da memorizzare nel dizionario; i valori delle chiavi degli elementi da memorizzare sono tutti diversi fra loro.

Per implementare un Dizionario abbiamo sostanzialmente 3 modi:

- Tabelle ad indirizzamento diretto;
- Tabelle hash;
- Alberi binari di ricerca.

Tabelle ad Indirizzamento Diretto

Non sono altro che array nel quale ogni indice corrisponde al valore della chiave dell'elemento da memorizzare in tale posizione.



Questa risulta la struttura dati più efficiente. Sia inserimento, ricerca che cancellazione avvengono in tempo costante. Tuttavia può essere utilizzata solo per un numero ristretto di dati. Altrimenti non sarebbe verificata l'ipotesi:

$$n \leq |U| = m$$

U può essere enorme, tanto da rendere impraticabile l'allocazione di un array A di sufficiente capienza; il numero delle chiavi effettivamente utilizzate può essere molto più piccolo di $|U|$, con rilevante spreco di memoria, in quanto la maggioranza delle posizioni dell'array A resta inutilizzata...

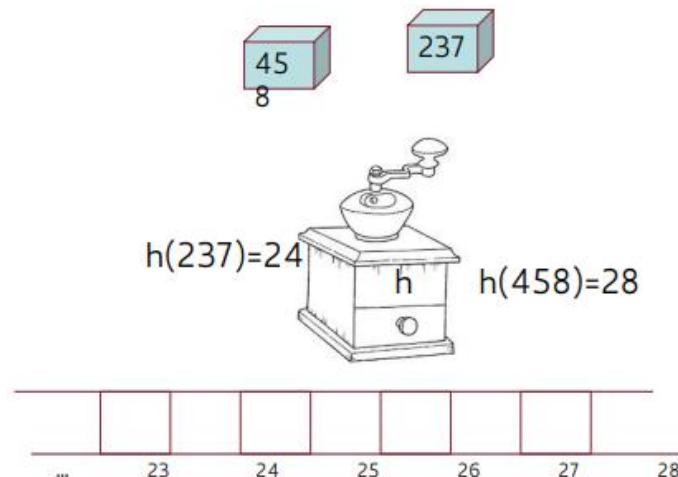
Tabelle Hash

Si usano quando U (cioè valori che le chiavi possono assumere) è molto grande, mentre l'insieme n delle chiavi effettive è molto più piccolo di U.

Idea: utilizzare un array di m posizioni.

Problema: non si può mettere direttamente in relazione la chiave con l'indice corrispondente, poiché le possibili chiavi sono molte di più rispetto agli indici.

Soluzione: Si definisce una opportuna funzione h, detta **funzione hash**, che viene utilizzata per calcolare la posizione di un elemento sulla base del valore della sua chiave



Problema: anche se le chiavi da memorizzare sono meno di m, non si può escludere che due chiavi $k_1 \neq k_2$ siano tali che $h(k_1) = h(k_2)$, per cui andrebbero memorizzate nella stessa posizione della tabella. Tale situazione viene chiamata **collisione** e NON c'è modo di evitarle. Possiamo solo evitarle il più possibile e, altrimenti, gestirle.

Una buona funzione hash deve rendere il più possibile **equiprobabili** i valori della funzione: tutti gli interi tra 0 ed $(m - 1)$ dovrebbero essere prodotti con uguale probabilità.

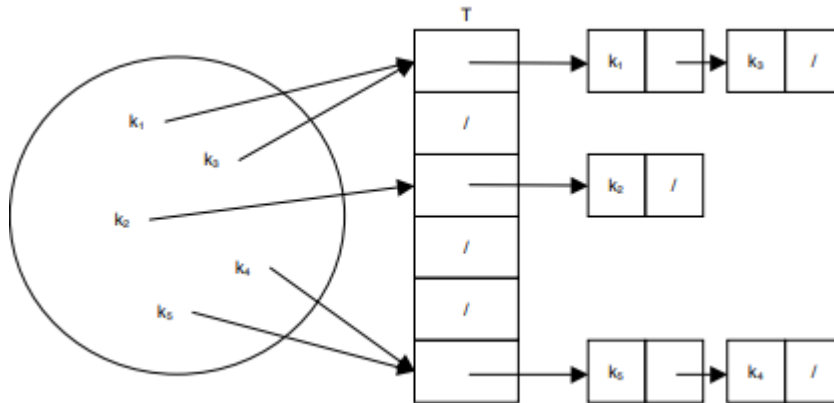
In altre parole, la funzione dovrebbe far apparire come "casuale" il valore risultante, disgregando qualunque regolarità della chiave. Inoltre, la funzione deve essere **deterministica**: se applicata più volte alla stessa chiave deve fornire sempre lo stesso risultato.

La situazione ideale è quella in cui ciascuna delle m posizioni della tabella è scelta deterministicamente con la stessa probabilità: ipotesi di uniformità semplice della funzione hash.

Sorvoliamo su come ottenere una funzione hash con l'ipotesi di uniformità semplice...

Liste di Trabocco

Prevedono di inserire tutti gli elementi le cui chiavi mappano nella stessa posizione in una lista concatenata, detta lista di trabocco.



Per ogni chiave calcolo il valore in hash e lo inserisco come testa di una lista. Se due chiavi hanno lo stesso valore in hash le inserisco in una lista uno come next dell'altro.

Inserimento

```
Insert_Liste_Trabocco (A; k: chiave da ins.)
    inserisci i dati dell'elemento di chiave k
    nella lista puntata da A[h(k)]
    return
```

Ricerca

```
Search_ListeTrabocco (A; k: chiave da cercare)
    ricerca k nella lista puntata da A[h(k)]
    if k è presente
        then: return puntatore all'elemento contenente k
    else: return None
```

Controllo se la chiave esiste nell'insieme delle chiavi. Se c'è la dovrò andare a cercare nella lista puntata corrispondente a quella chiave. La ricerca con liste puntate richiede di scorrere tutti gli elementi.

Cancellazione

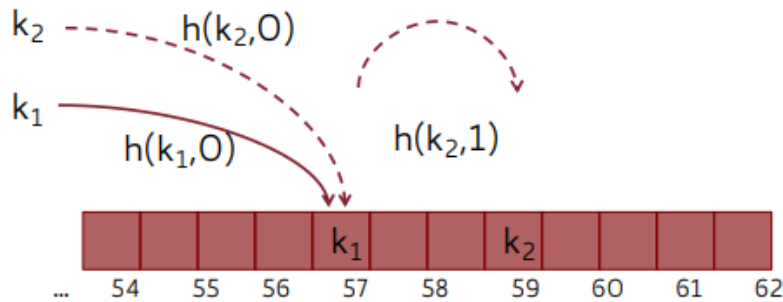
```
Delete_ListeTrabocco (A; k: chiave da canc.)
    cancella i dati dell'elem. di chiave k
    dalla lista puntata da A[h(k)]
    return
```

Indirizzamento Aperto

Questa tecnica prevede di inserire tutti gli elementi direttamente nella tabella, senza far uso di strutture dati aggiuntive. Essa è applicabile quando $m \gg \text{numero } n \text{ di elementi da memorizzare}$ (quindi il fattore di carico non è mai maggiore di 1) ed $|U| \gg m$

Idea: invece di avere una lista per ogni posizione dell'array, inseriamo solo all'interno dell'array stesso, calcolando (...) la sequenza delle posizioni da esaminare. La funzione hash sarà quindi più complessa, perché ora dipende da 2 parametri: la chiave k e il numero di collisioni già trovate.

Nota: la sequenza prodotta deve contenere una permutazione di tutti gli indici in modo da garantire che tutte le celle siano considerate.



Inserimento

Se la posizione iniziale relativa alla chiave k è occupata, si scandisce la tabella fino a trovare una posizione libera nella quale l'elemento con chiave k può essere memorizzato. La scansione è guidata da una sequenza di funzioni hash ben determinata: $h(k, 0), h(k, 1), \dots, h(k, m-1)$

Ricerca

Si scandisce la tabella mediante la stessa sequenza di funzioni hash utilizzata per l'inserimento fino ad incontrare l'elemento cercato o una casella vuota (= l'elemento non è presente). Costo nel caso di ricerca senza successo: Come per l'inserimento: nel caso peggiore $O(n)$, ma nel caso medio (quando la funzione hash gode di uniformità semplice) $1/(1-\alpha)$ dove $\alpha = n/m$ è il fattore di carico della tabella.

Costo nel caso di ricerca con successo: nell'ipotesi di uniformità semplice, il numero di accessi atteso per una ricerca con successo è: $1 + \log_2 \frac{1}{1-\alpha} + 1$ dove $\alpha = n/m$ è il fattore di carico.

Esempio: con una tabella piena al 50% il numero di accessi atteso è meno di 3,387; con una tabella piena al 90% il numero di accessi atteso è meno di 3,67; tutto questo indipendentemente dalle dimensioni della tabella!

ATTENZIONE: Anche se questa implementazione dei dizionari garantisce grande efficienza nel caso medio, NON si può dire che il costo di ogni operazione sia COSTANTE nel caso peggiore. Questo deve essere tenuto in conto quando si calcola il costo computazionale di un algoritmo che faccia uso dei dizionari. (veramente importante se nel compito utilizziamo i dizionari come strutture dati, compresi quelli già implementati di python, ed effettuiamo operazioni di ricerca o cancellazione).

Cancellazione

se si lascia la casella vuota, non si possono recuperare gli elementi memorizzati in caselle successive (una ricerca è senza successo quando si incontra una casella vuota...). • se si marca la casella con un valore deleted, il costo computazionale della ricerca non dipende più esclusivamente dal fattore di carico ma anche dal numero delle posizioni precedentemente occupate da elementi e successivamente marcate. Perciò di solito la cancellazione non è supportata con l'indirizzamento aperto.

Hashing Doppio

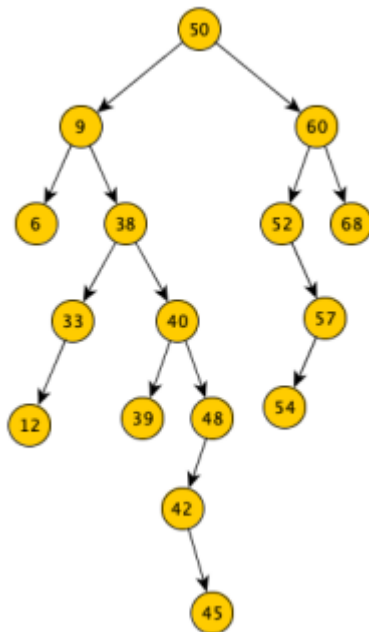
Una tecnica usata nella pratica per generare funzioni hash adatte all'indirizzamento aperto è l'**hashing doppio**.

Idea: Usare due diverse funzioni hash, una per determinare l'accesso iniziale alla tabella e l'altra per determinare il passo di scansione: $h(k, i) = [h_1(k) + i \cdot h_2(k)] \bmod m$, $i = 0, 1, \dots, m - 1$. Funziona perché, se le due funzioni sono ben progettate, è estremamente improbabile che due chiavi $k_1 \neq k_2$ producano una collisione su entrambe le funzioni hash (nel qual caso le due chiavi scandirebbero l'intera tabella nello stesso modo, situazione indesiderabile).

Alberi Binari di Ricerca (ABR)

Un albero binario di ricerca (ABR) è un albero in cui vengono mantenute le seguenti proprietà: • ogni nodo contiene una chiave • il valore della chiave contenuta in ogni nodo è maggiore della chiave contenuta in ciascun nodo del suo sottoalbero sinistro (se esiste) • il valore della chiave contenuta in ogni nodo è minore della chiave contenuta in ciascun nodo del suo sottoalbero destro (se esiste)

Esempio:



Gli ABR sono strutture dati che supportano tutte le operazioni già definite sugli insiemi dinamici (più alcune altre).

Operazioni di interrogazione:

- **Search**(T, k): restituisce un puntatore all'elemento con chiave di valore k in T se questo è presente, None altrimenti;
- **Minimum**(T), **Maximum**(T): restituisce un puntatore all'elem. in T con min/max chiave;
- **Predecessor**(T,p), **Successor**(T,p): restituisce un puntatore all'elem. in T con la chiave che precederebbe/seguirebbe in una sequenza ordinata la chiave contenuta nel nodo puntato da p.

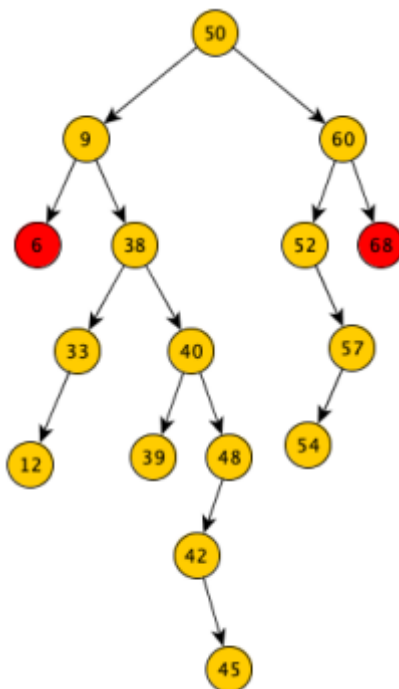
Operazioni di manipolazione:

- **Insert**(T, k): inserisce un elem. di chiave k in T;
- **Delete**(T, p): elimina da T l'elem. puntato da p.

Utilizzo ABR

Un ABR può essere usato sia come dizionario che come coda di priorità: il minimo è sempre nel nodo più a sinistra, il massimo in quello più a destra.

N.B. il nodo più a sinistra non necessariamente è una foglia: può anche essere il nodo più a sinistra che non ha un figlio sinistro; analoga considerazione per il nodo più a destra.



Elencare le in Ordine Crescente

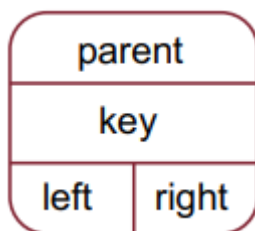
Tenendo conto di quanto visto fino ad ora, cioè che l'elemento minimo si trova più a sinistra e l'elemento massimo più a destra, ci accorgeremo che quello che dobbiamo fare è sostanzialmente leggere l'albero da sinistra verso destra. Come visto nelle altre lezioni è molto semplice: basta infatti effettuare delle visite in order, che hanno proprio la particolarità di vedere prima a sinistra, poi nel nodo e poi a destra.

Dunque un ABR può anche essere visto come una struttura dati su cui eseguire un algoritmo di ordinamento, costituito di due fasi:

- inserimento di tutte le n chiavi da ordinare in un ABR, inizialmente vuoto;
- visita in-ordine dell'ABR appena costruito.

Il costo computazionale di tale algoritmo è: $T(n) = \text{Costo}(\text{costruzione ABR}) + (n)_{\text{Costo(visita)}}$

Nel seguito assumiamo che l'ABR sia memorizzato tramite record a puntatori e che ciascun nodo abbia, oltre ai soliti puntatori ai figli destro e sinistro, anche un puntatore al padre (utile per risalire agli antenati)



ABR - Ricerca

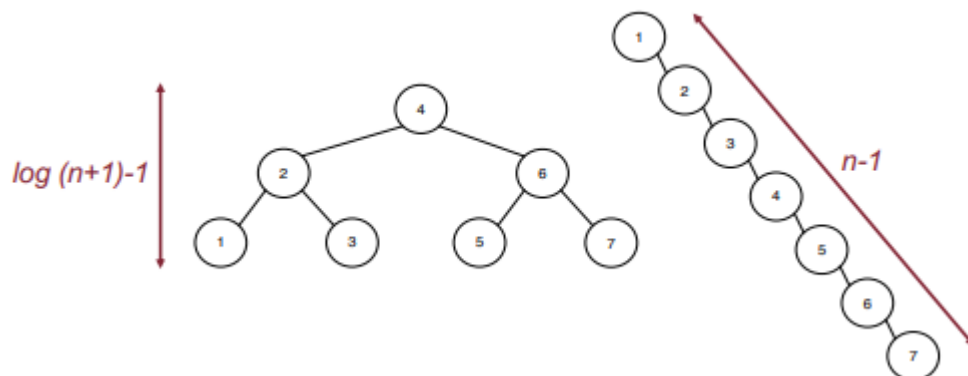
Concettualmente simile alla ricerca binaria. Si esegue una discesa dalla radice controllando se l'elemento che sto cercando è minore del nodo vado a sinistra; se è maggiore vado a destra. Continuo finché non lo trovo oppure arrivo alle foglie (in quel caso non è stato trovato)

```
def ABR_searchRic(p,k):
    if (p == None) or (p->key == k):
        return p
    if (k < p->key):
        return ABR_searchRic(p->left, k)
    else:
        return ABR_searchRic(p->right, k)
```

Considerando che al caso generale avremo costo $T(h) = t(h-1) + \Theta(1)$, avremo che il costo costituzionale sarà $\Theta(h)$ dove h è l'altezza dell'albero. Di fatto richiamando la funzione ricorsiva sul nodo subito sotto di noi stiamo diminuendo l'altezza di 1.

Considerando che la ricerca su un ABR ricorda molto da vicino la ricerca binaria (che ha costo computazionale $O(\log n)$) si potrebbe pensare che anche qui abbiamo costo $O(\log n)$... Sbagliato!!

L'ABR NON include fra le sue proprietà alcuna limitazione sulla sua altezza. Pertanto potremo avere in un caso altezza logaritmica e in altri altezza lineare...



Analogamente al Quicksort, si dimostra che il comportamento degli ABR nel caso medio è molto più vicino al caso migliore del caso peggiore: dato un ABR costruito in modo casuale con n chiavi, vale:

Teorema. L'altezza attesa di un albero binario di ricerca costruito in modo casuale con n chiavi tutte distinte è $O(\log n)$.

ABR – Inserimento

Guardo la radice. Se la chiave che devo inserire è più piccola vado a sinistra. Se è più grande vado a destra e continuo finché non trovo un nodo none che verrà sostituito con il nuovo valore da inserire.

Per quanto riguarda il codice questo sarà simile a quanto visto per l'inserimento in coda delle liste. Avremo due puntatori, che si muovono uno davanti all'altro. Quando il primo punterà a none saremo arrivati alla fine dell'albero, e cambieremo il campo next() del puntatore superiore inserendo il nuovo elemento.

```

def ABR_insert (p,k):
    y,x=None,p          #y punta sempre al padre di x
    z=NodoABR(k)
    while (x != None)    #discesa alla prima pos. disponib.
        y = x
        if z->key < x->key:
            x = x->left
        else:
            x = x->right
    if (y==None)         #se albero inizialmente vuoto
        p = z
    else:
        if (z->key < y->key):
            y->left = z
        else:
            y->right = z
    z->parent = y
    return p

```

Se quello che sto cercando < x

Vado a sinistra

Altrimenti vado a destra

ciclo eseguito al max h volte quindi $T(h)=O(h)$

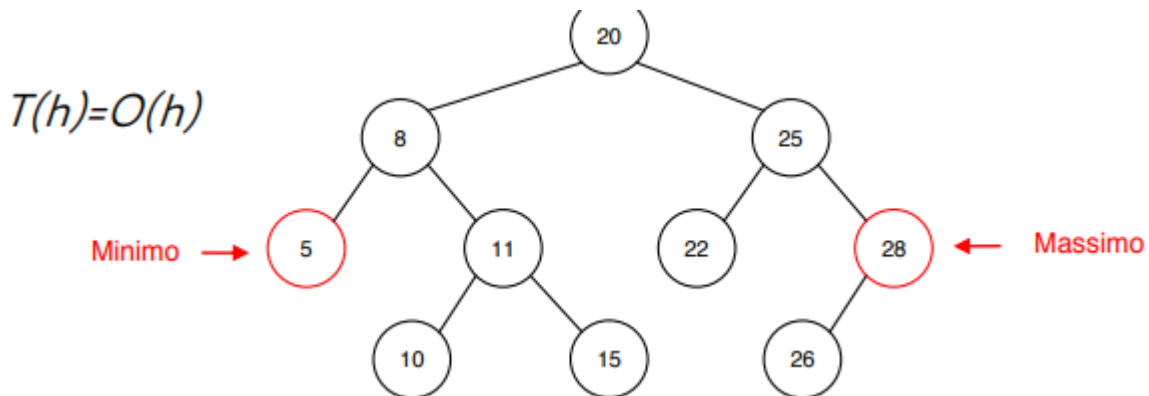
#collegam. padre - nodo da inser.

#p potrebbe essere cambiato

Per quanto riguarda il costo computazionale questo sarà uguale a quello della ricerca. Di fatto il codice è praticamente identico. Semplicemente aggiungiamo quella parte di if per invertire gli indici dove serve. Ma ha tutto costo $\theta(1)$.

ABR - Ricerca Min/Max

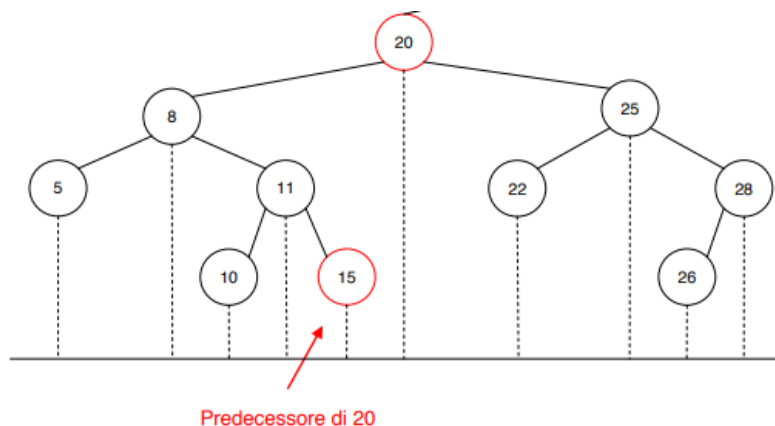
Il minimo (massimo) si trova nel nodo più a sinistra (destra), quindi per trovarlo si scende sempre a sinistra (destra) a partire dalla radice. Ci si ferma quando si arriva a un nodo che non ha figlio sinistro (destro): quel nodo contiene il minimo (massimo).



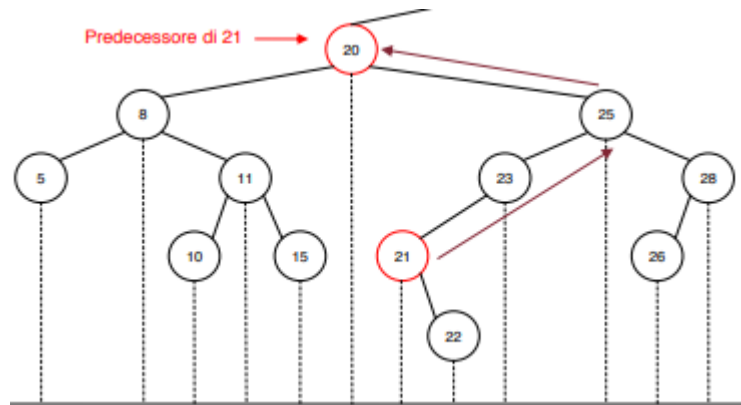
ABR – Predecessore/Successore

Il predecessore è il più grande dei numeri più piccoli.

I numeri più piccoli sono tutti situati a sinistra del nodo su cui applichiamo predecessor(). Ed il più grande è situato nel nodo più a destra. Pertanto il predecessore di un qualunque nodo è il max (elemento più a destra) del sottoalbero sinistro.



Tuttavia non è detto che il sottoalbero sinistro esista... se sto controllando una foglia come facciamo? Dobbiamo risalire l'albero finché non troviamo un "padre" più piccolo del nostro nodo. Quello sarà il nostro predecessore.

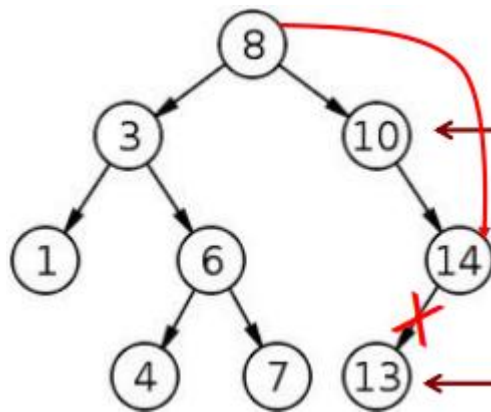


ABR – Cancellazione

Così come nelle altre strutture dinamiche la cancellazione potrebbe risultare complicata perché dobbiamo assicurare le proprietà della struttura dati rimangano invariate nonostante la cancellazione.

Per eliminare un nodo in un ABR:

- **Caso 1:** se il nodo è una foglia lo si elimina, ponendo a None l'opportuno campo nel nodo padre;
- **Caso 2:** se il nodo ha un solo figlio lo si "cortocircuita", cioè si collegano direttamente fra loro suo padre e il suo unico figlio, indipendentemente che sia destro o sinistro;



- **Caso 3:** Se la chiave da eliminare è in un nodo con due figli va riaggiustato l'albero dopo l'eliminazione per evitare che si disconnetta. Riaggiustare l'albero significa trovare un nodo da collocare al posto del nodo che va eliminato, così da mantenere l'albero connesso e da garantire il mantenimento della proprietà fondamentale degli ABR. Tale nodo può quindi essere solamente il predecessore o il successore del nodo da eliminare. Una volta scambiati sarà più semplice cancellare il nodo, perché a quel punto potrà avere solo 1 o 0 figli.

16/05/2024

Un ABR di altezza h contenente n nodi supporta le operazioni fondamentali dei dizionari in tempo $O(h)$, ma purtroppo non si può escludere che h sia $O(n)$, con conseguente degrado delle prestazioni. Viceversa, tutte le operazioni restano efficienti se si riesce a garantire che l'altezza dell'albero sia piccola, in particolare sia $O(\log n)$. Per raggiungere tale scopo esistono varie tecniche, dette di bilanciamento.

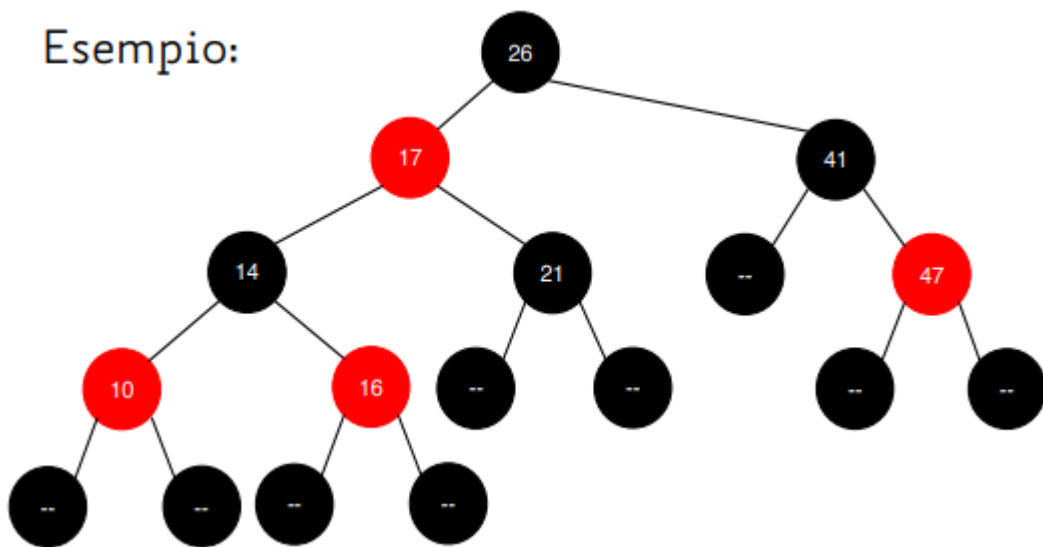
Le tecniche di bilanciamento sono tutte basate sull'idea di riorganizzare la struttura dell'albero se essa, a seguito di un'operazione di inserimento o di eliminazione di un nodo, viola determinati requisiti. In particolare, il requisito da controllare è che, per ciascun nodo dell'albero, l'altezza dei suoi due sottoalberi non sia "troppo differente".

Alberi Rosso-Neri

Un albero rosso-nero (RB-albero) è un ABR i cui nodi hanno un campo aggiuntivo, il colore, che può essere solo rosso o nero. Alla struttura si aggiungono foglie fittizie (che non contengono chiavi) per fare in modo che **tutti i nodi** “veri” dell’albero abbiano **esattamente due figli**. Dunque gli alberi Rosso-Neri hanno TUTTE le proprietà degli ABR più alcune proprietà aggiuntive:

1. ciascun nodo è rosso o nero;
2. ciascuna foglia fittizia è nera;
3. se un nodo è rosso i suoi figli sono entrambi neri;
4. ogni cammino da un nodo a ciascuna delle foglie del suo sottoalbero contiene lo stesso numero di nodi neri.
5. La radice è sempre nera

Esempio:



Nota: per non sprecare spazio in memoria. l'insieme delle foglie fittizie può essere sostituito da un unico oggetto, cui puntano tutti i puntatori che indicano una foglia fittizia (sentinella).

Altezza e Principali Operazioni degli Alberi RB

Proprietà: Il più corto cammino radice-foglia è lungo più del doppio del più lungo cammino radicefoglia. Infatti:

- il numero di nodi neri è uguale lungo tutti i cammini radice-foglia (proprietà 4);
- il numero di nodi rossi lungo un cammino non può essere maggiore del numero di nodi neri lungo lo stesso cammino (proprietà 3);
- un cammino che contiene il max numero di nodi rossi non può essere lungo più del doppio di un cammino composto solo di nodi neri.

La b-altezza di un nodo x ($bh(x)$ =black height di x) è il numero di nodi neri sui cammini dal nodo x (non incluso) alle foglie sue discendenti (è uguale per tutti i cammini, proprietà 4). b-altezza di un RB-albero = $bh(r)$

NOTA: se un albero rispetta le proprietà 1-4 ma ha la radice rossa (e quindi non rispetta la proprietà 5), basta cambiare il colore della radice in nero per ottenere un RB-albero valido. La sua b-altezza risulterà aumentata di 1.

Teorema: Un RB-albero con n nodi interni ha altezza $h \leq 2 \log(n + 1)$.

Questo ci garantisce che le operazioni di, ricerca di una chiave, massimo o del minimo, predecessore o del successore siano tutte identiche a quelle per gli ABR ma che siano tutte eseguite con un costo computazionale $O(\log n)$.

Questo però **NON vale per le operazioni di inserimento e cancellazione**, perché vanno mantenute le proprietà di RB-albero!! Dopo un inserimento o una cancellazione, la struttura dell'albero può dover essere riaggiustata, in termini di colori assegnati ai nodi o collocazione dei nodi nell'albero.

Rotazioni

Le rotazioni permettono di ripristinare in tempo $O(\log n)$ le proprietà dell'RB-albero dopo un inserimento o una cancellazione. Le rotazioni possono essere destre o sinistre e sono operazioni locali che non modificano l'ordinamento delle chiavi secondo la visita in-ordine.

```
def RBA_rotaz_sinistra (p,a): #a=punt. al nodo su cui si ruota
```

```
    b = a->right
```

```
    a->right = b->left
```

```
    if a->right != None:
```

```
        a->right->parent = a
```

```
    b->left = a
```

```
    b->parent = a->parent
```

```
    if a->parent==None:
```

```
        p = b
```

```
    else:
```

```
        if a==a->parent->left:
```

```
            a->parent->left = b
```

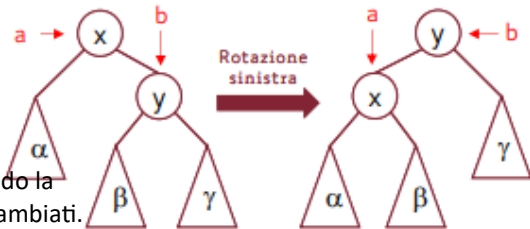
```
        else:
```

```
            a->parent->right = b
```

```
    a->parent = b
```

```
    return p
```

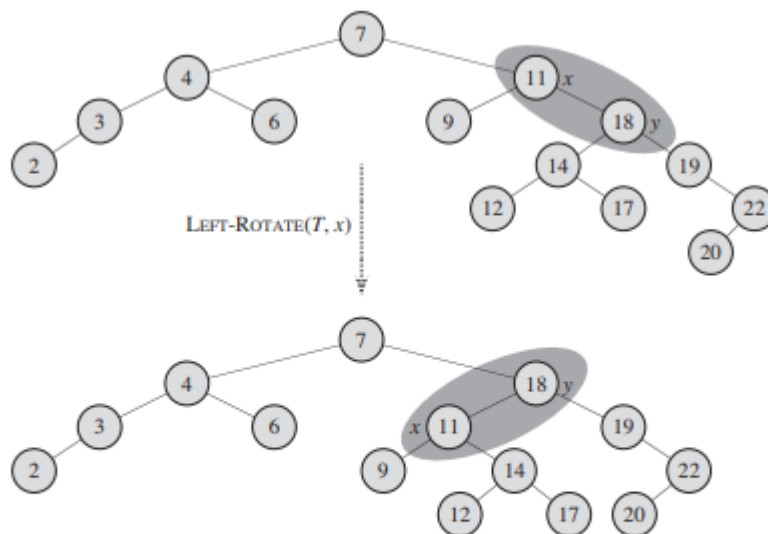
Se a non ha padre è la radice. Svolgendo la rotazione vogliamo che a e b siano scambiati. Quindi impostiamo b come radice.



Costo computazionale: $\theta(1)$.

(La rotazione a destra è analoga)

Esempio:



Nota: la visita in-order prima e dopo la rotazione produce lo stesso elenco di chiavi!

Inserimento

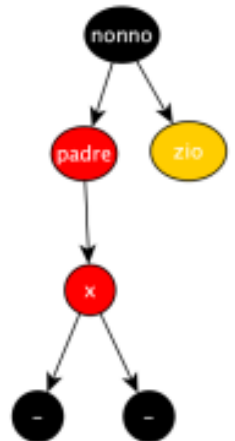
Passo Preliminare: Quando si inserisce l'elemento seguendo le regole dell'inserimento in un ABR, attribuendo **sempre colore rosso** al nuovo nodo.

Le regole 1. (ciascun nodo è rosso o nero) e 2. (le foglie fittizie sono nere) rimangono sempre vere. La regola 4. (ogni cammino da un nodo a ciascuna foglia ha lo stesso numero di nodi neri) rimane vera, visto che il nuovo nodo è sempre rosso. Le uniche regole che possono essere infrante sono:

- La 5. (la radice è sempre nera), che però si risolve cambiando colore
- La 3. (entrambi i figli di un nodo rosso sono neri), se il nuovo nodo (rosso) potrebbe essere inserito come figlio di un nodo rosso... in questi casi come risolviamo?

Passo di Aggiustamento: Va eseguito solo se il nuovo nodo è stato inserito come figlio (rosso) di un nodo rosso. Facciamo prima di tutto chiarezza sulla situazione attuale al momento dell'inserimento:

- il nonno esiste (perché un nodo rosso non può essere radice)
- il nonno è nero (perché un nodo rosso non può avere un figlio rosso)
- lo zio esiste ma non sappiamo il colore (perché ogni nodo interno ha due figli)

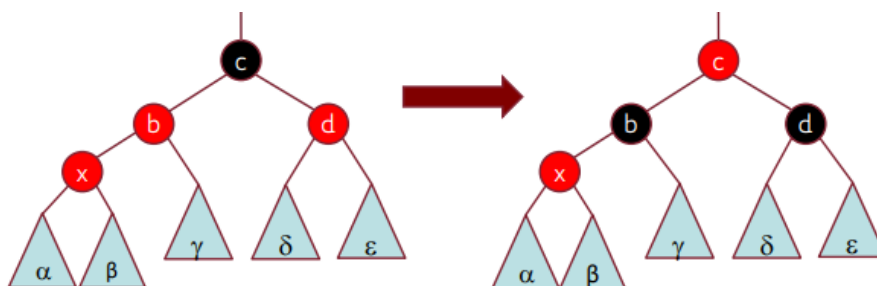


Avremo quindi 4 possibilità:

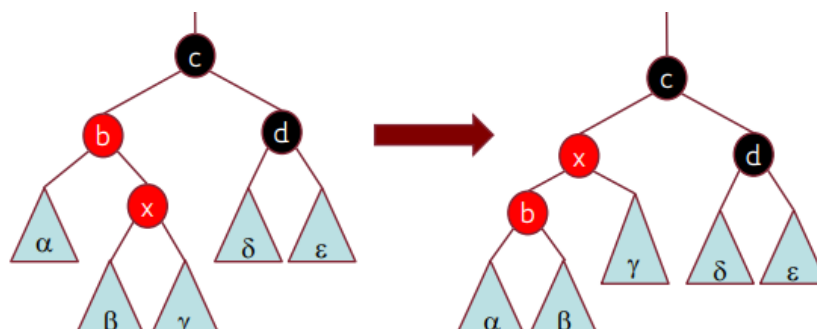
- **Caso 0:** il nodo inserito è radice. Avremo però radice rossa che aggiustiamo cambiando colore



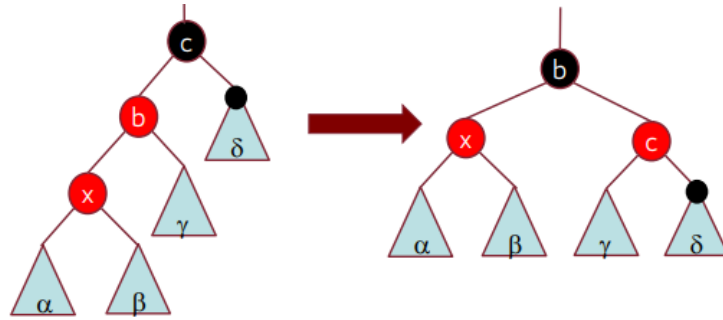
- **Caso 1:** il nodo X inserito è figlio di un nodo rosso e lo zio è rosso. Si risolve colorando neri il padre e lo zio di x e impostando rosso il nonno di x. Ora, o la violazione è stata eliminata (fine inserimento), o è stata portata più in alto (di 2 livelli) e possiamo risolverla con uno degli altri casi.



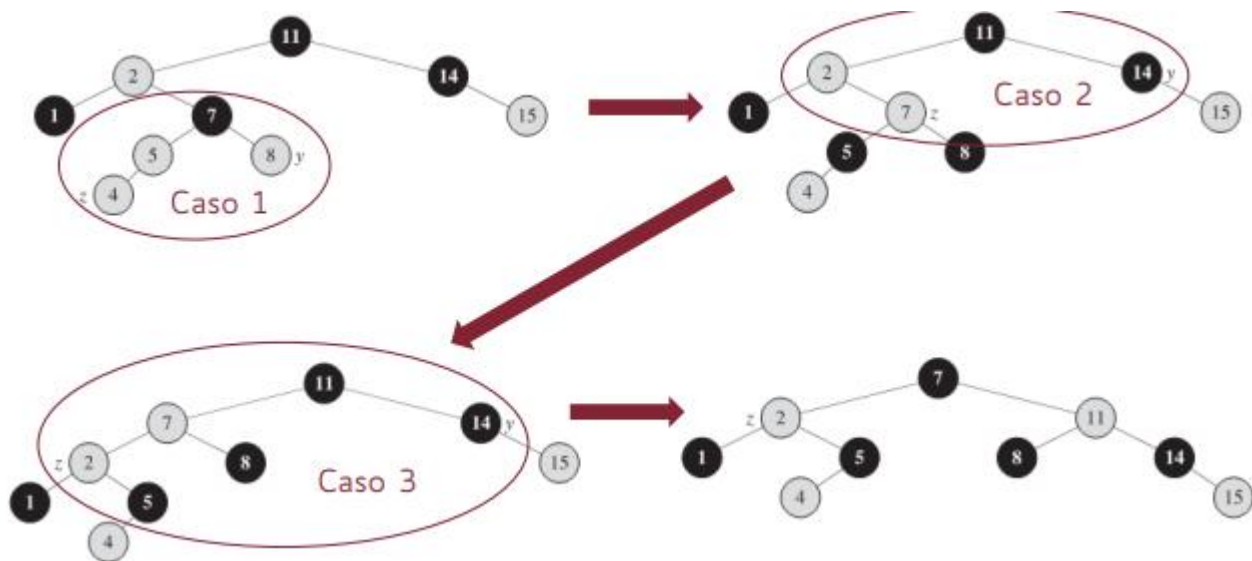
- **Caso 2:** il nodo X inserito è figlio destro di un nodo rosso e lo zio è nero. Si effettua una rotazione a sinistra imperniata sul padre di x, e questo conduce sempre al Caso 3.



- **Caso 3:** il nodo X inserito è figlio sinistro di un nodo rosso e lo zio è nero. Si effettua una rotazione e si cambiano alcuni colori. Ciò garantisce la soluzione della violazione.



Esempio:



NOTA: il problema si può propagare verso l'alto! quando la propagazione raggiunge la radice, l'altezza nera dell'albero cresce. Il Costo Computazionale di questa cosa è $O(n)$.